Fast cut cell quadratures and robust contouring algorithms

Samuel F. Potter

Robert McNeel & Associates
Seattle, WA

Who am I?

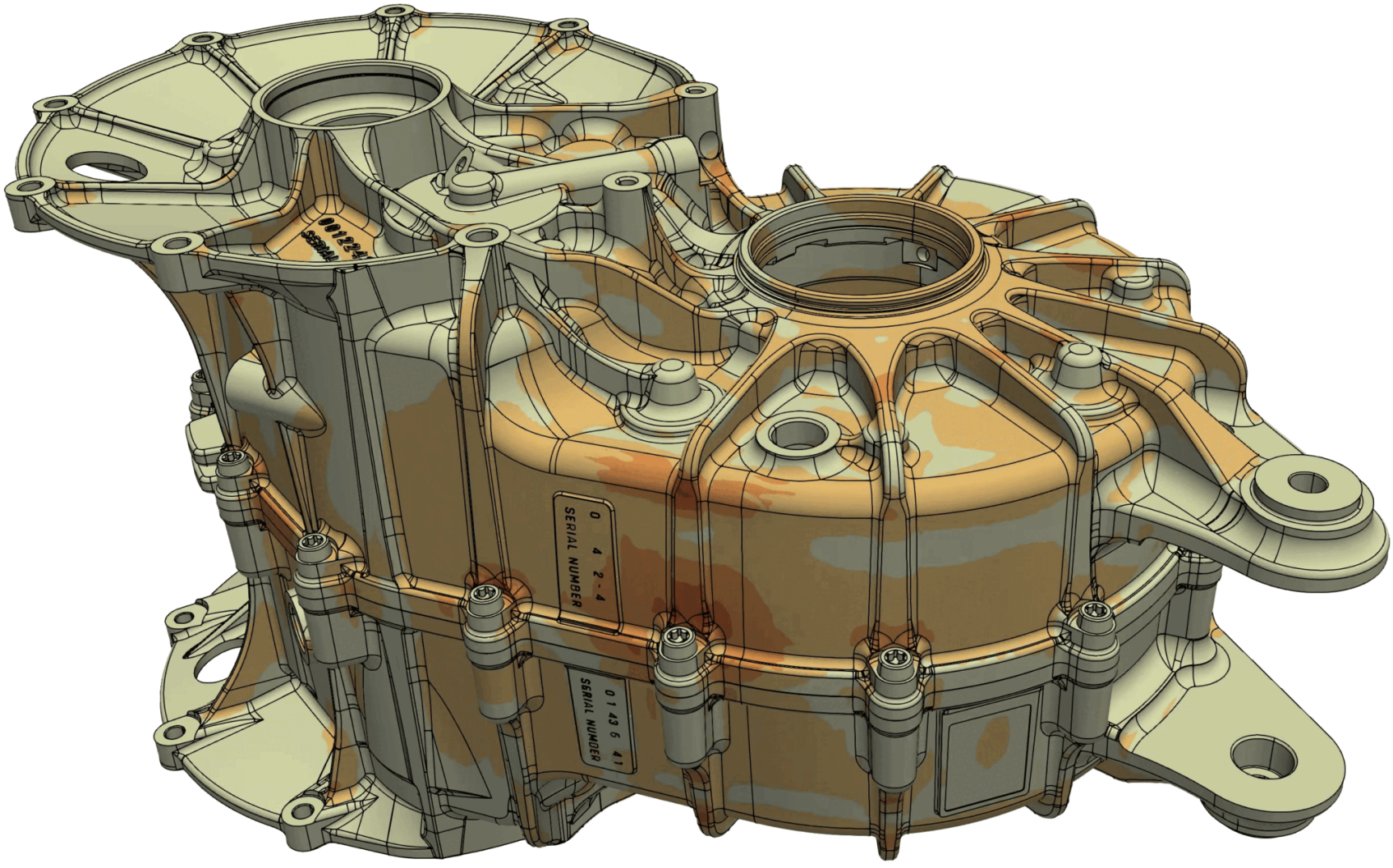
- Me: https://sampotter.github.io
- Currently employed by Robert McNeel & Associates (makers of **Rhinoceros 3D** (www.rhino3d.com)
- **Previously**: research scientist at Coreform, LLC (**first part of this talk**)
- **Previously**: CI at NYU w/ Leslie Greengard and Mike O'Neil
- **Previously**: CS PhD from UMD w/ Masha Cameron

McNeel

- There are no titles at McNeel, but I'm effectively a research engineer.
- I have very specific problems that I need to solve.
- Instead of trying to find novel problems and solutions, I'm trying to find the **best** solution. Most robust, most accurate, fastest.
- That often means research and novelty because many existing solutions are bad ideas or are inadequate.
- "Old literature full of treasure." - Yi Ma
- Working on modernizing their CAD kernel: faster, more accurate, more robust... (**second part of this talk**)

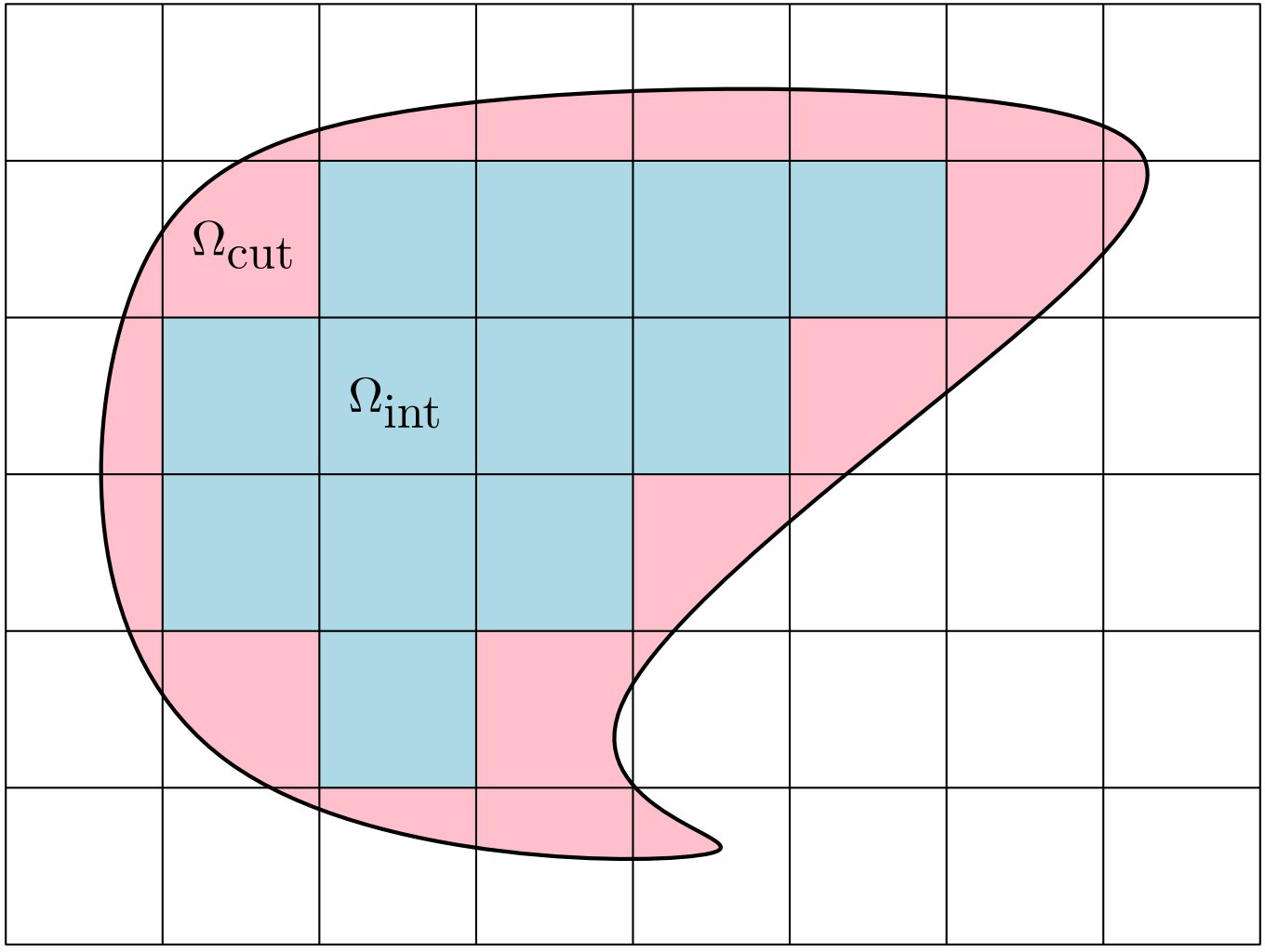
Fast cut cell quadratures

Coreform



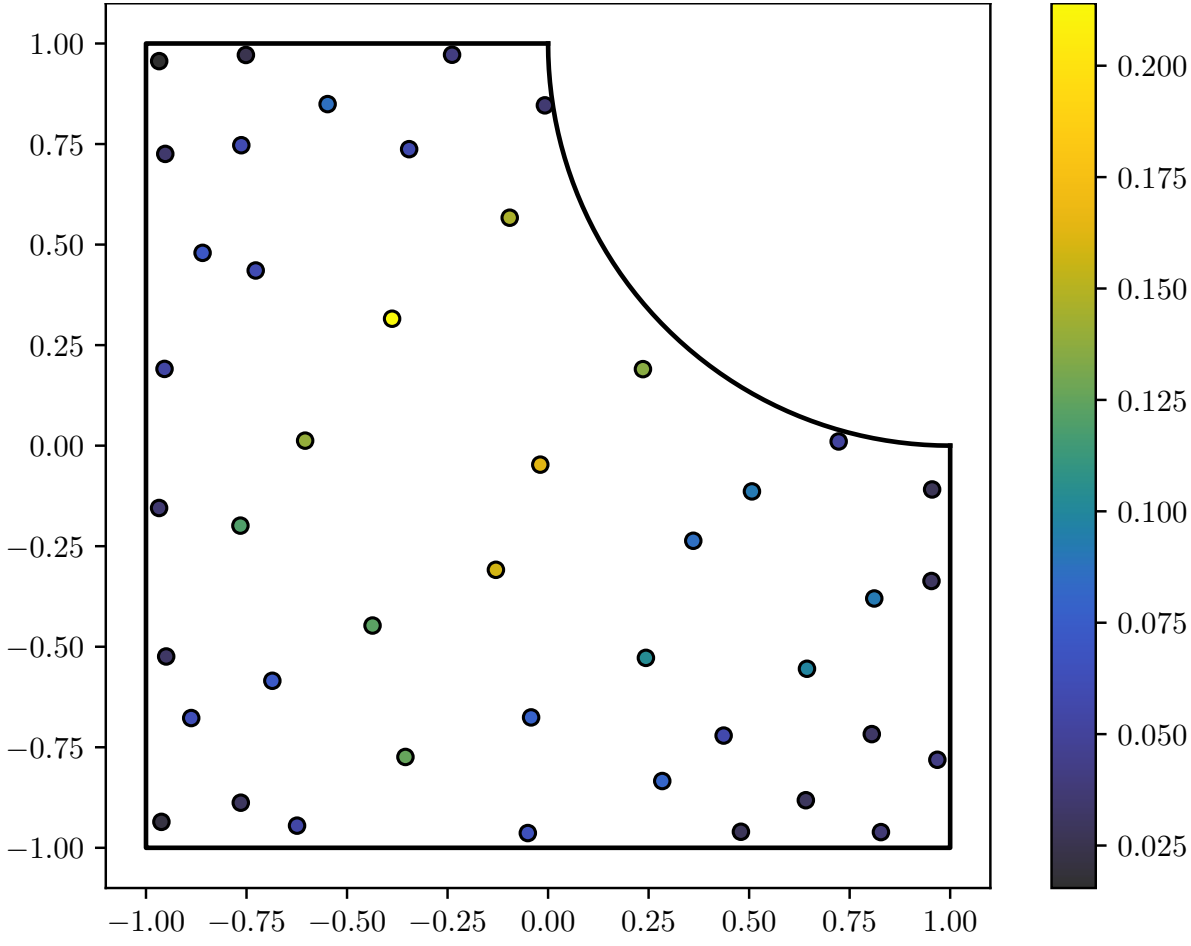
Cut cell quadratures

Useful algorithms for on-the-fly construction of "Gaussian"-ish cut cell quadratures...



Cut cell quadratures

Useful algorithms for on-the-fly construction of "Gaussian"-ish cut cell quadratures...



The beast

- Choose basis $\{\phi_\alpha\}$ (α a multi-index) for multivariate space of polynomials $\mathbb{P}$
- For order n quadrature, find weights and nodes satisfying:

$$Q[\phi] = \sum_{i=1}^{n} w_i \phi_\alpha(x_i) = \int_\Omega \phi_\alpha, \qquad \forall \alpha$$

- **Nonlinear, underdetermined**: can solve with Gauss-Newton with good initial guess
- **Linear** if node positions are fixed (**but how to choose nodes?**)

The efficiency of a quadrature rule

Number of **degrees of freedom** of $\mathcal{Q}_n$ is:

$$\mathrm{dof}(\mathcal{Q}_n) = (d+1)n \qquad \text{(for } w_i \text{ and } x_i\text{)}$$

and the **efficiency** of $\mathcal{Q}_n$ by:

$$\mathrm{eff}(\mathcal{Q}_n) = \frac{\dim \mathbb{P}_{m,d}}{\mathrm{dof}(\mathcal{Q}_n)}$$

where $0 \leq \mathrm{eff}(\mathcal{Q}_n) \leq 1$.

- Gauss-Legendre: $\mathrm{eff} = 1$
- Clenshaw-Curtis: $\mathrm{eff} = 1/2$
- Tensor Gauss-Legendre: $\mathrm{eff} \sim 2^d/(d+1)! \sim 0$ as $n \to \infty$

Existence and stability of positive quadratures

Theorem (Tchakaloff)

For $\Omega \subseteq \mathbb{R}^d$ compact, exists ***positive*** *quadrature with order not greater than $m = \dim \mathbb{P}$ with nodes in Ω.*

Theorem (Huybrechs 2009)

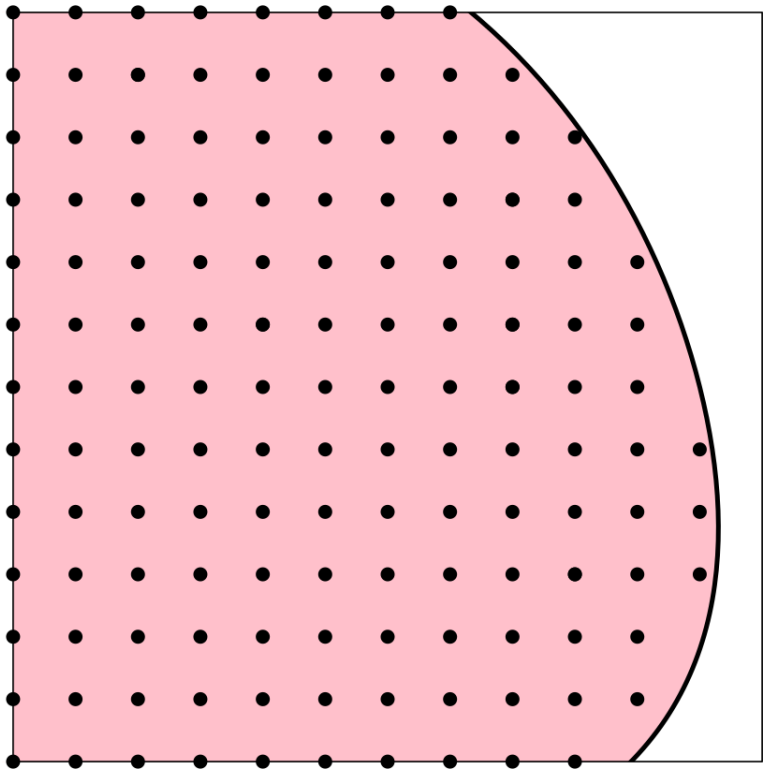
Let $\kappa(\mathcal{Q}_n) = \sum_i |w_i|$, let f a function be integrated, and $\tilde{f}$ an approximation of f with $|f - \tilde{f}| \leq \epsilon$ pointwise. Then:

$$\left| \sum_i w_i f(x_i) - \sum_i w_i \tilde{f}(x_i) \right| \leq \epsilon \kappa(\mathcal{Q}_n).$$

Strictly larger *if $\mathcal{Q}_n$ isn't positive.*

Discretizing the cut cell

1. Map cut cell bounding box onto $[-1, 1]^d$
2. Discretize the box into a uniform, isotropic grid with grid spacing $h > 0$
3. Discard all points that lie strictly outside Ω



Discretizing the cut cell

- Nodes: $x_1, \ldots, x_n$
- Basis matrix for moment-matching equations:

$$V = \begin{bmatrix} \phi_1(x_1) & \cdots & \phi_m(x_1) \\ \vdots & \ddots & \vdots \\ \phi_1(x_n) & \cdots & \phi_m(x_n) \end{bmatrix} \tag{1}$$

- Moment vector:

$$I \in \mathbb{R}^m \quad \text{st} \quad I_\alpha = \int_\Omega \phi_\alpha(x) dx \tag{2}$$

- Linear moment-matching (**underdetermined**):

$$V^\top w = I \tag{3}$$

Quadrature toolbox: overview

Goal: using our oversampled grid, find sparse, nonnegative solution, subset of positive weights. Lots of approaches:

- Least squares
- Steinitz elimination
- Nonnegative least squares
- Basis pursuit (won't cover, equivalent to NNLS)
- Xiao-Gimbutas elimination
- Ryu-Boyd (linear prog.)

Quadrature toolbox: least squares rules

Least squares quadrature:

- Solve:

$$\text{minimize} \quad \frac{1}{2}\|V^{\mathsf{T}} w - I\|_2^2$$

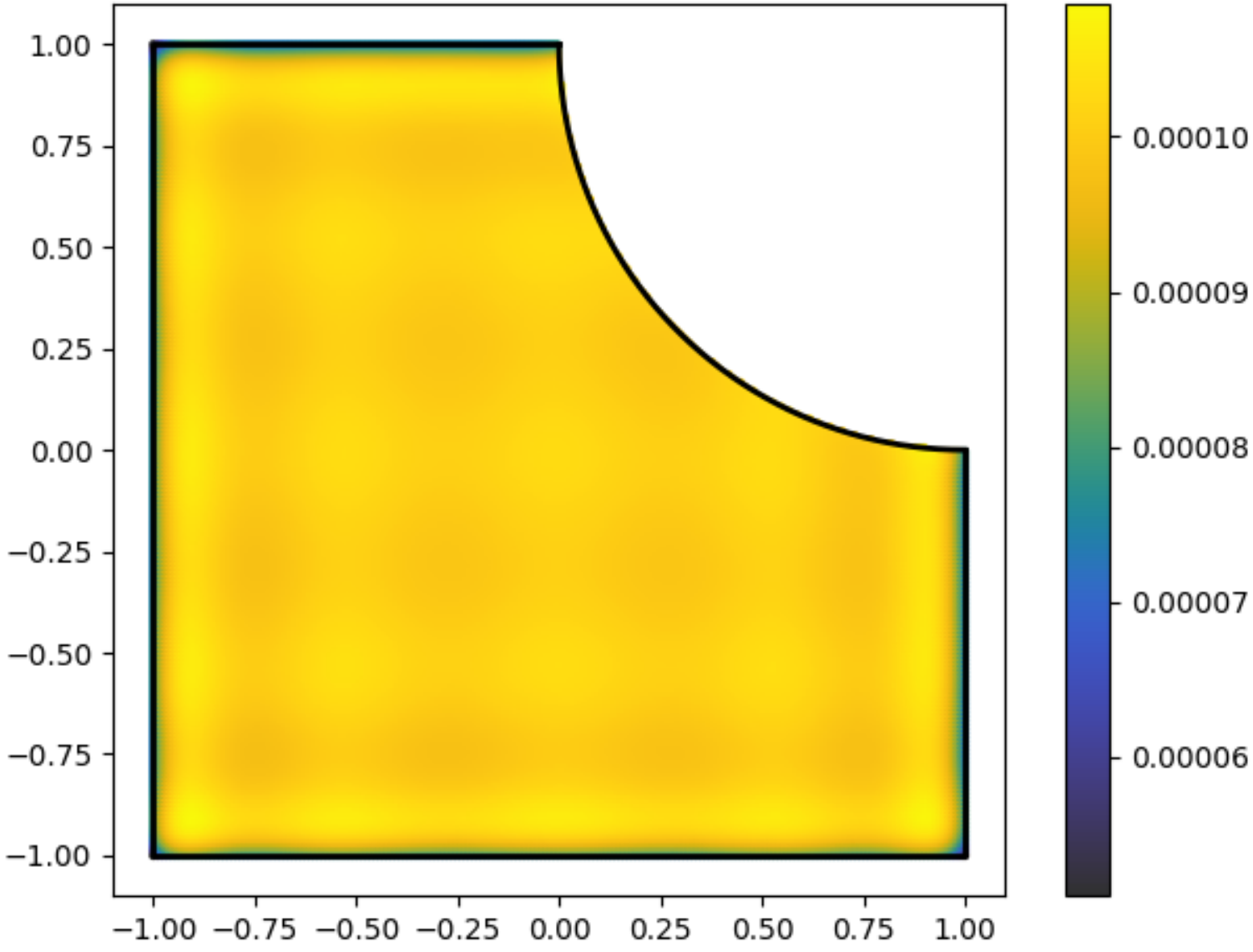
  using LU decomposition or QR.

Theorem (Existence: Huybrechs 1D, me *n*D)

There exists a positive integer n_0 such that for $n \geq n_0$ the resulting least squares quadrature is positive.

Problem: $w > 0$ at all nodes.

Quadrature toolbox: least squares rules



(For 32,121 points...)

Quadrature toolbox: Steinitz elimination

Can we remove unnecessary quadrature nodes?
Yes, look in the nullspace of $V^\top$ to find shift Δw so that:

$$V^\top(w + \Delta w) = I, \qquad w + \Delta w \geq 0. \tag{4}$$

Theorem (Steinitz)

Let $\mathcal{Q}_n$ be a positive quadrature that integrates $\mathbb{P}$ on Ω such that $n \geq m = \dim \mathbb{P}$. Then there exists a positive quadrature of order m w/ subset of nodes of $\mathcal{Q}_n$.

Consequence: positive least squares rule exists $\Longrightarrow$ order m rule exists, proof is constructive.

Quadrature toolbox: NNLS rules

Nonnegative least squares quadrature:

- Solve:

$$\begin{aligned} \text{minimize} \quad & \frac{1}{2}\|V^{\mathsf{T}} w - I\|_2^2 \\ \text{subject to} \quad & w \geq 0 \end{aligned}$$

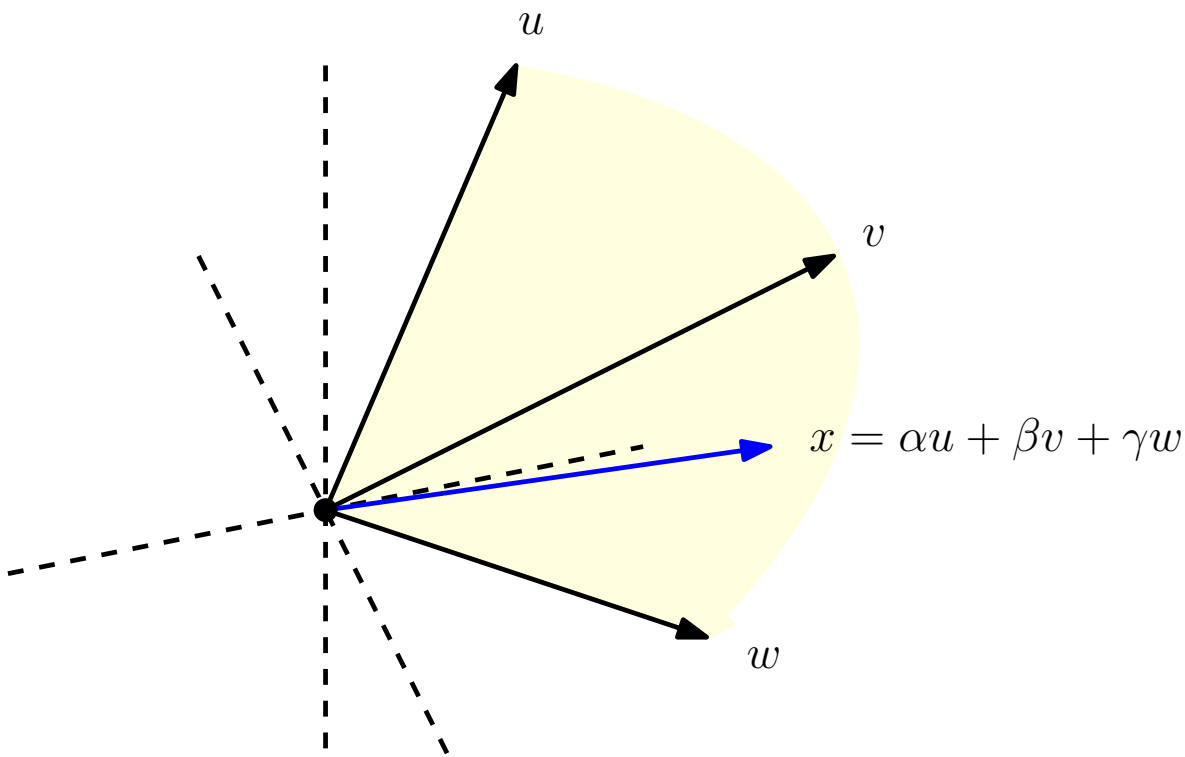
Undeterdetermined convex optimization problem, can have infinite number of solutions.

Can always find m-sparse solution, but solution may not have zero residual

Quadrature toolbox: NNLS rules

Theorem (Carathéodory)

If $x \in cone(A) \subseteq \mathbb{R}^m$, then x can be written as the conical combination of m points in A.



Note: $V^\top w = I$ is a conic combination! But Carathédory theorem isn't constructive.

Quadrature toolbox: NNLS rules

Problem: solution biased by algorithm when nonunique, need to use right algorithm.

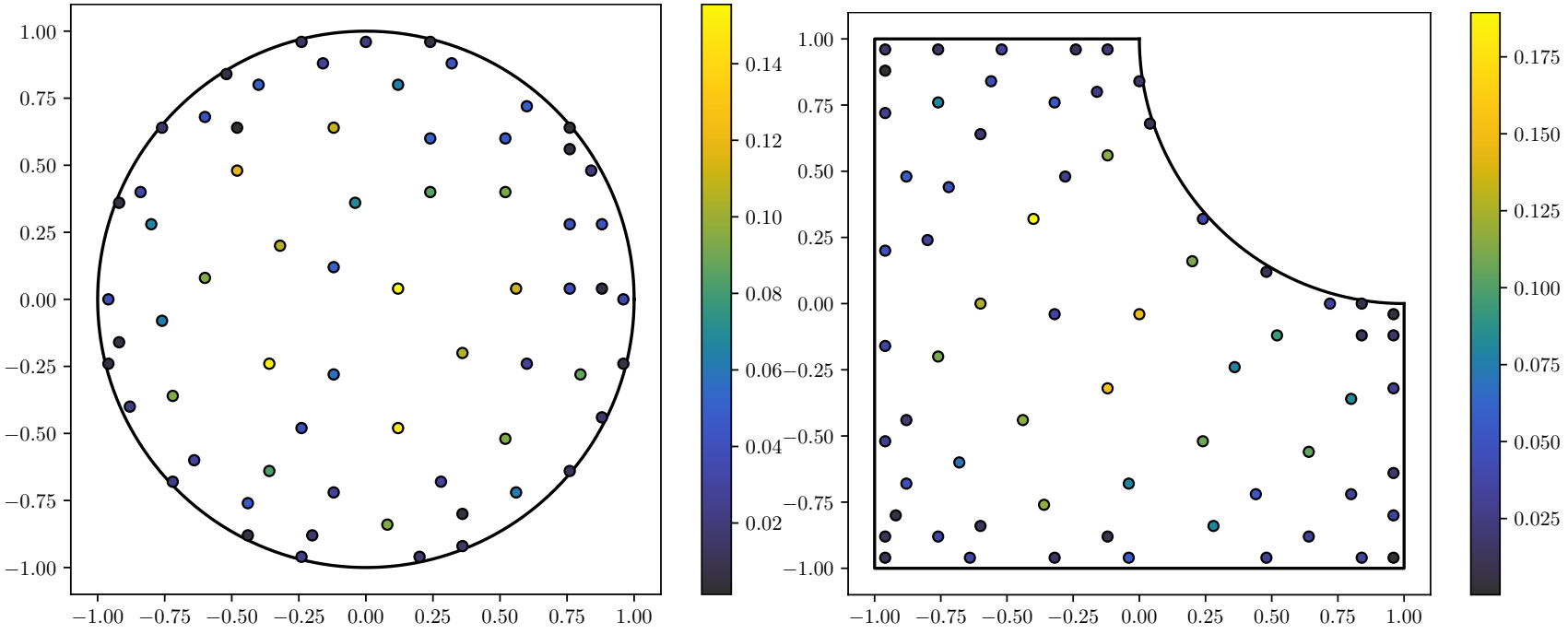
Algorithm NNLS:

1. Use the Lawson-Hanson algorithm (i.e. primal active set method for quadratic program with linear inequality constraints) to solve NNLS.
2. Return the subset of weights and nodes for which $w_i > 0$.

Typically relaxes one quadrature node at a time, converging to an extreme point on the feasible set.

Quadrature toolbox: NNLS rules

What does this look like?



Quadrature toolbox: Xiao-Gimbutas elimination

A concern: In general, we expect the solution of either NNLS or BP to be M-sparse. This only achieves:

$$\mathrm{eff}(\mathcal{Q}) = \frac{1}{d+1}. \tag{5}$$

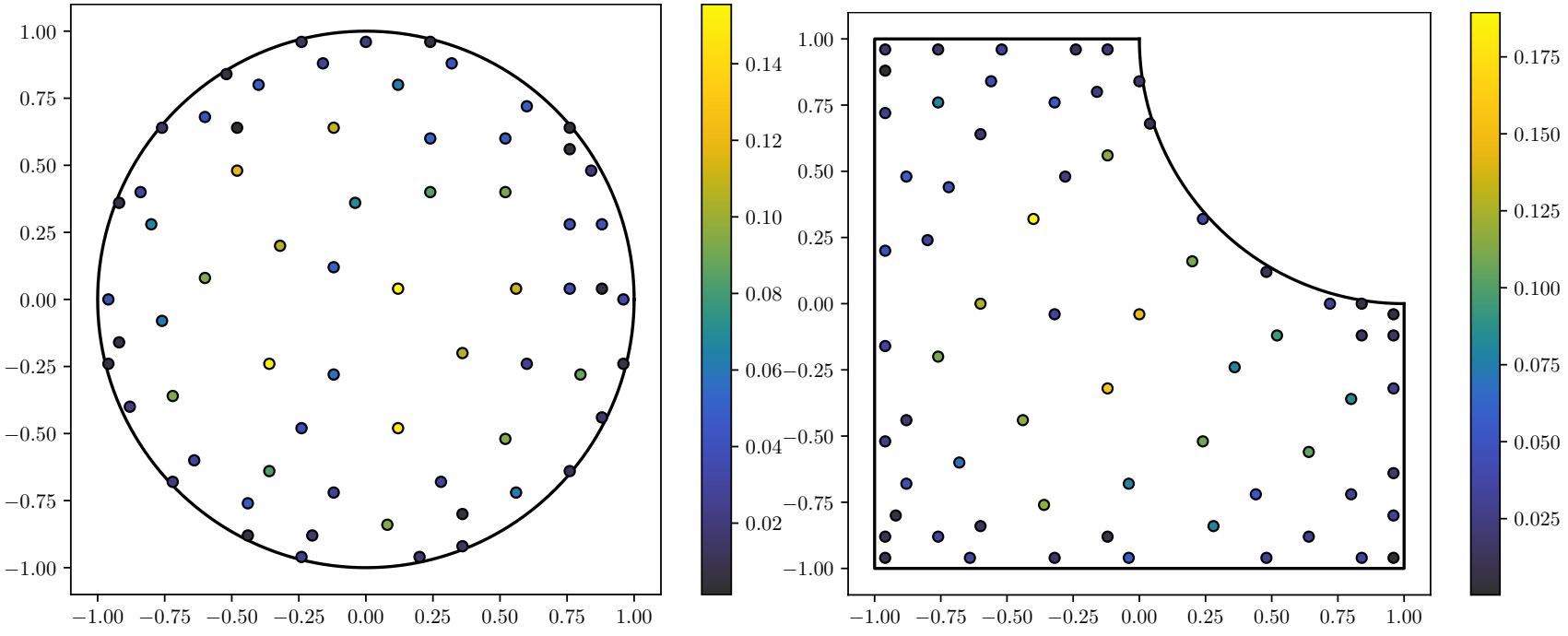
Nonlinear node elimination: Xiao and Gimbutas propose the following method for removing nodes from a quadrature which differs from Steinitz' (linear).

Algorithm XG:

1. Compute $\sigma_j = w_j \sum_\alpha \phi_\alpha(x_j)^2$ for each j.
2. Delete node j from the quadrature.
3. Run Gauss-Newton on the result.

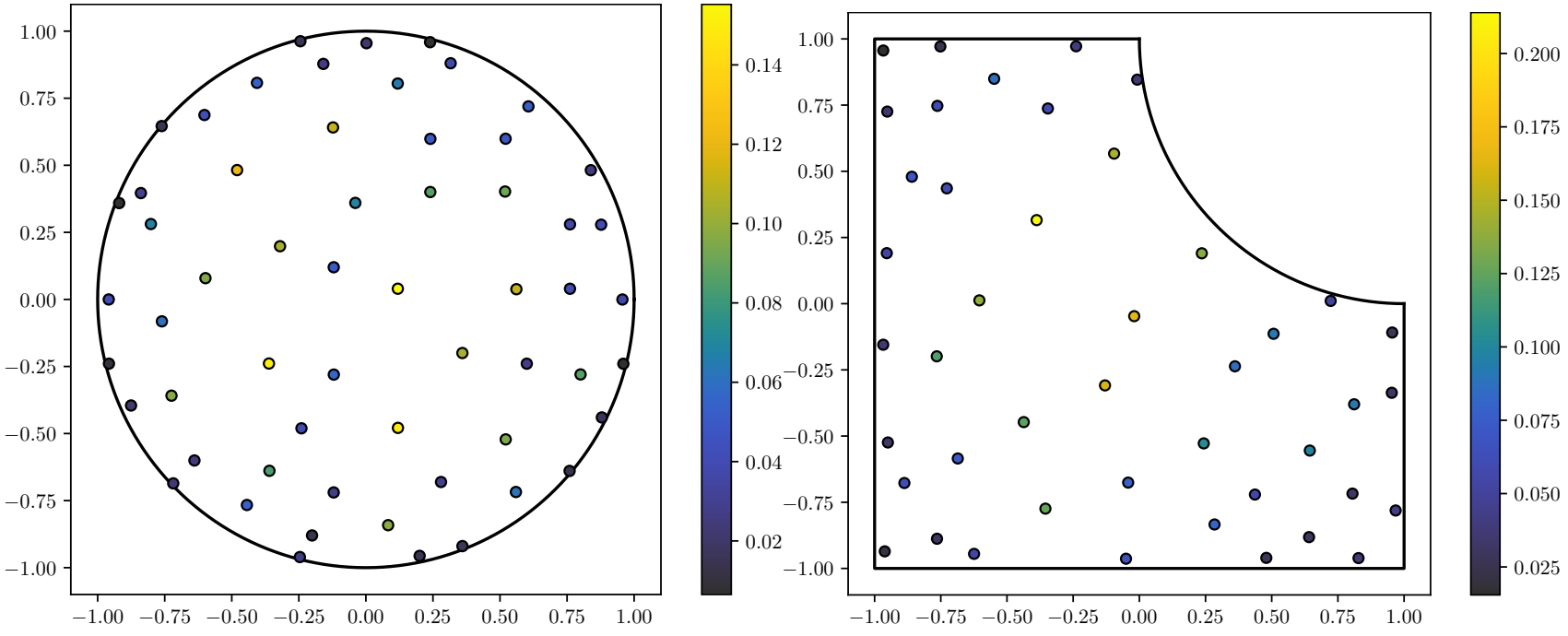
Quadrature toolbox: Xiao-Gimbutas elimination

What does this look like?



Quadrature toolbox: Xiao-Gimbutas elimination

What does this look like?



Quadrature toolbox: Ryu-Boyd

Ryu and Boyd consider the semi-infinite linear program:

$$\begin{aligned} \text{minimize}_{\mu} \quad & \int_{\Omega} r d\mu \\ \text{subject to} \quad & \int_{\Omega} \phi_\alpha d\mu = \int_{\Omega} \phi_\alpha(x) dx \qquad \forall \alpha \\ & \mu \geq 0 \end{aligned}$$

where optimization is over **all nonnegative measures supported on** Ω, and r is a "residual" function (dealer's choice!).

Ryu and Boyd prove that on $[-1, 1]$, with $r(x) = x^{2n}$ and $\phi_i(x) = x^i$ $(0 \leq i < 2n)$, Gaussian-Legendre quadrature is the unique sol'n of this problem.

Quadrature toolbox: Ryu-Boyd

Lay down uniform grid and discretize to get the LP:

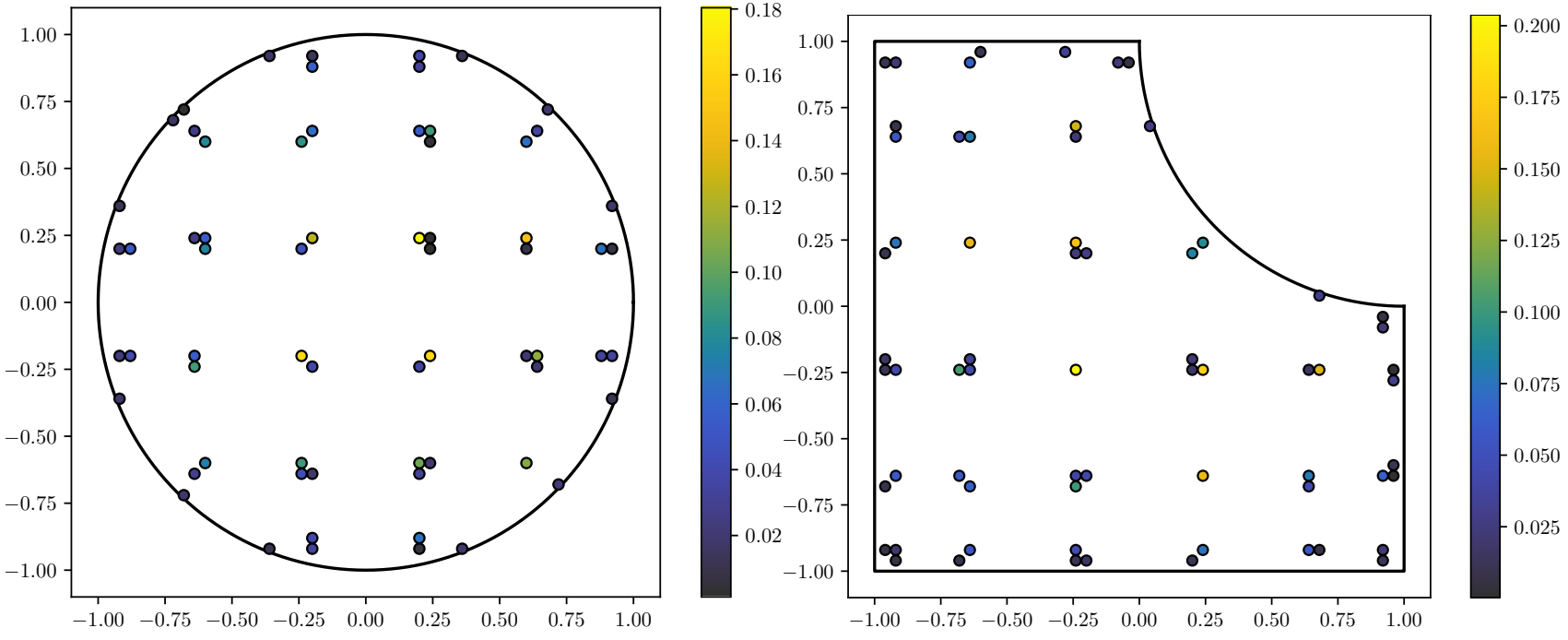
$$\begin{array}{lll} \text{minimize} & r^{\mathsf{T}} w & \\ \text{subject to} & V^{\mathsf{T}} w = I & \text{(RB)} \\ & w \geq 0. & \end{array}$$

where r is the "residual", a parameter.

We use $r(x, y) = x^{2n} + y^{2n}$ throughout, but choice of residual needs more work.

Quadrature toolbox: Ryu-Boyd

What happens if we throw RB at a black box LP solver?



Generically results in eff$(\mathcal{Q}) = 1/(d+1)$ quadrature... but look at the nodes...

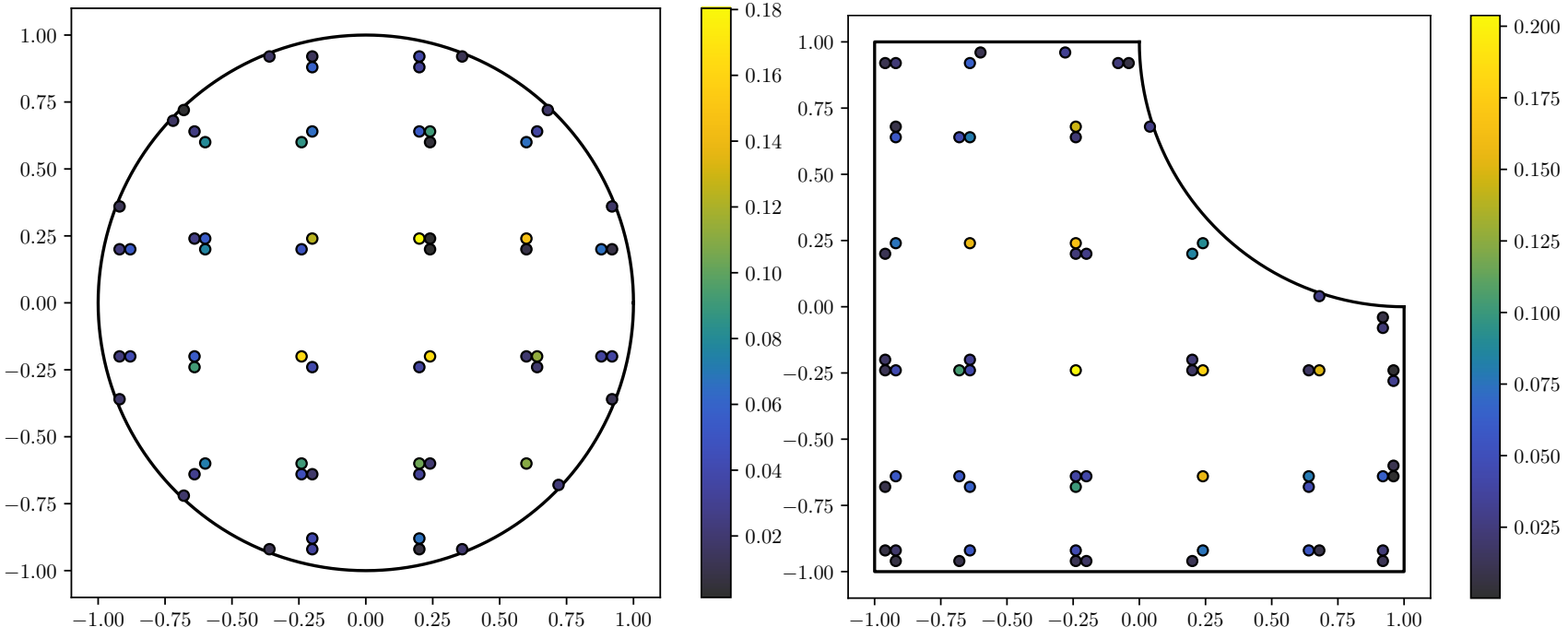
Quadrature toolbox: Ryu-Boyd

Algorithm RB:

- Solve RB using black-box LP solver.
- Identify clusters of points.
- Replace each cluster with a single quadrature point:
  - Let the new weight w_{clust} be the sum of the weights.
  - Let the new node be the convex combination of the old nodes using the normalized quadrature weights for each point, i.e. $x_{\text{clust}} = \sum_{j \in \text{clust}} (w_j / w_{\text{clust}}) x_j$.
- Optimize the "clustered" quadrature using Gauss-Newton.

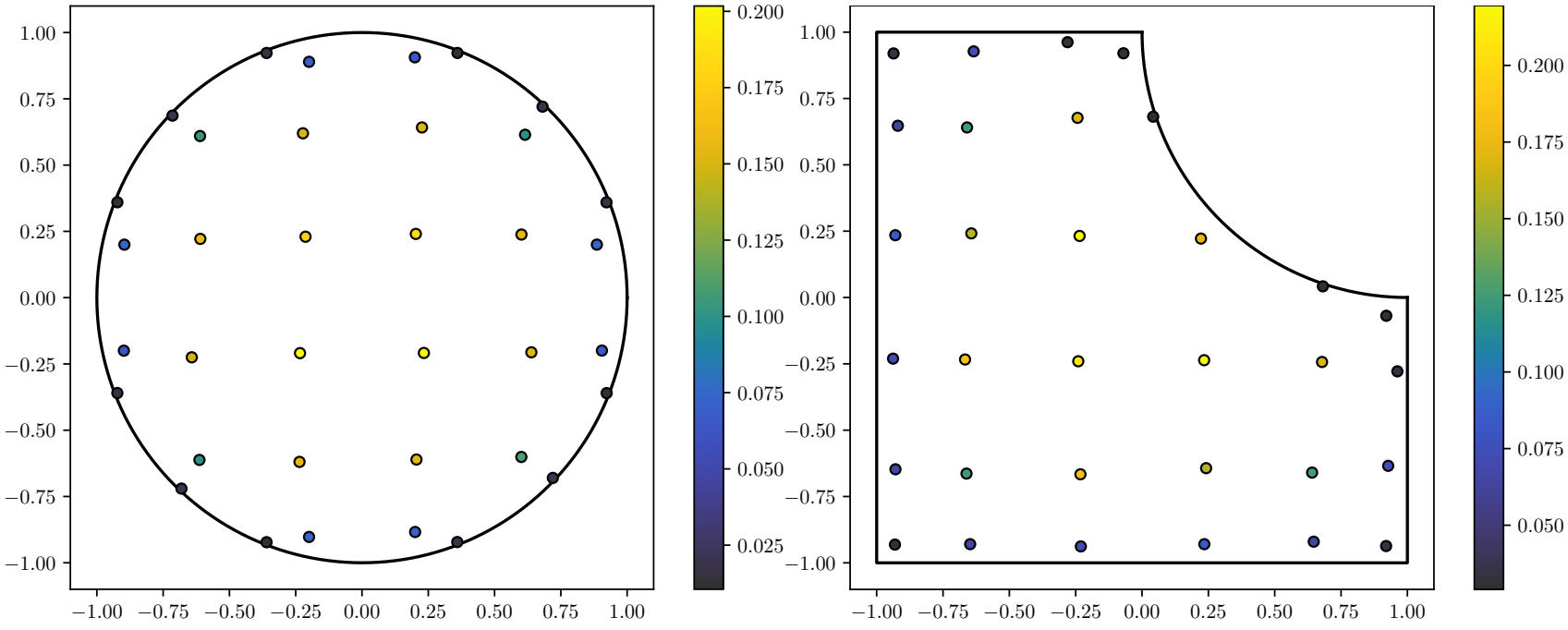
Quadrature toolbox: Ryu-Boyd

What does this look like?



Quadrature toolbox: Ryu-Boyd

What does this look like?



Quadrature toolbox: primal-dual RB

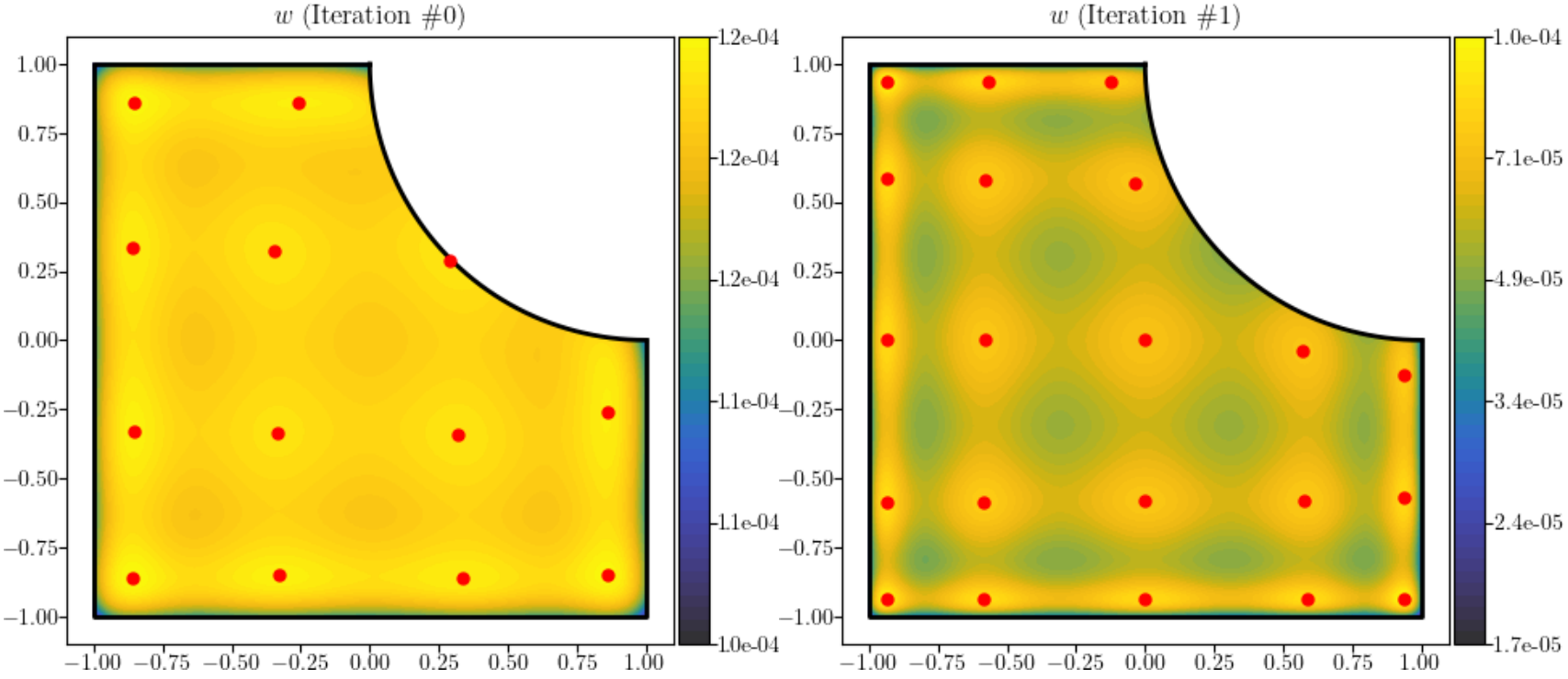
What if we look under the hood of our LP solver? Two standard approaches: **simplex** and **interior point**.

An interior point method for solving Ryu-Boyd is a **curious choice**, because by construction it produces iterates which have w_i for every i at each step!

But the result converges pointwise to an M-sparse solution...?

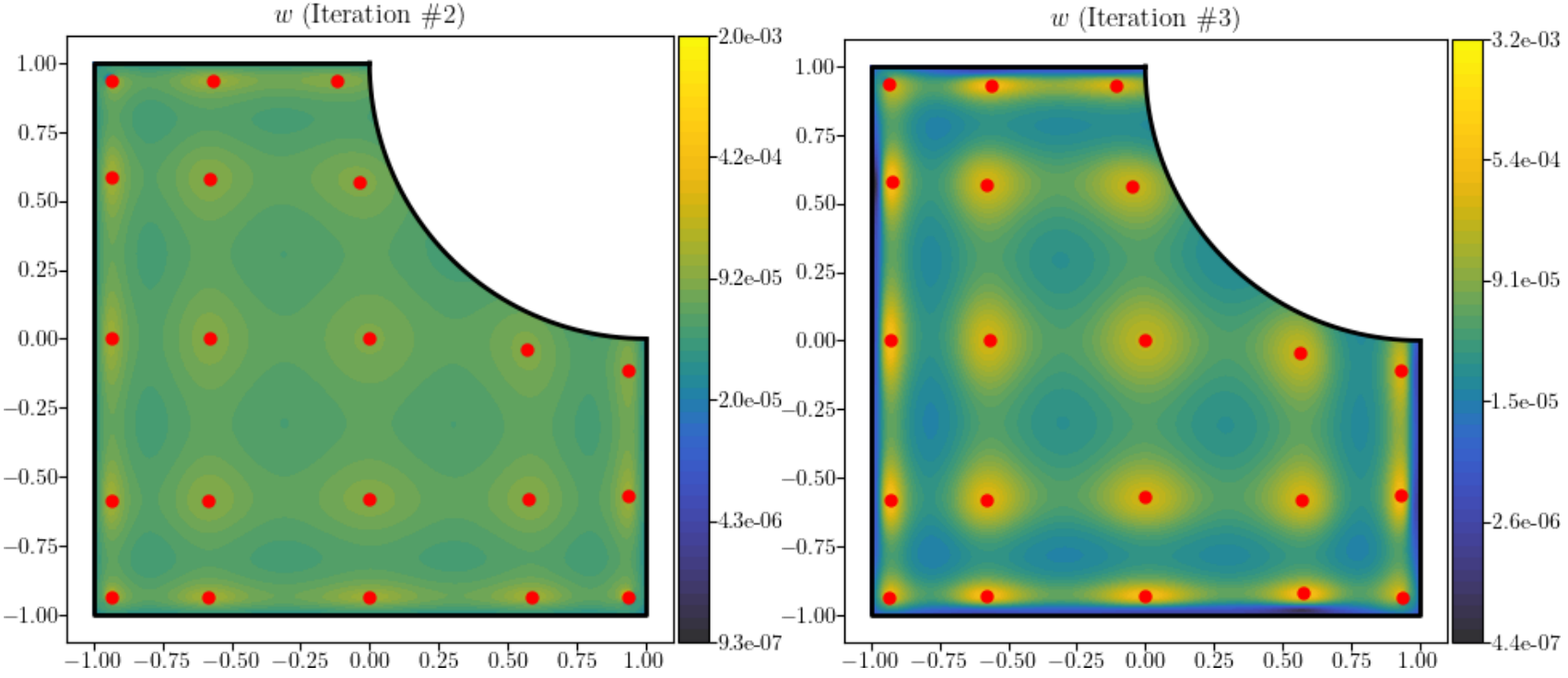
Quadrature toolbox: primal-dual RB

It works! And after only a single step. Just need to find local optima and re-optimize using Gauss-Newton after selection:

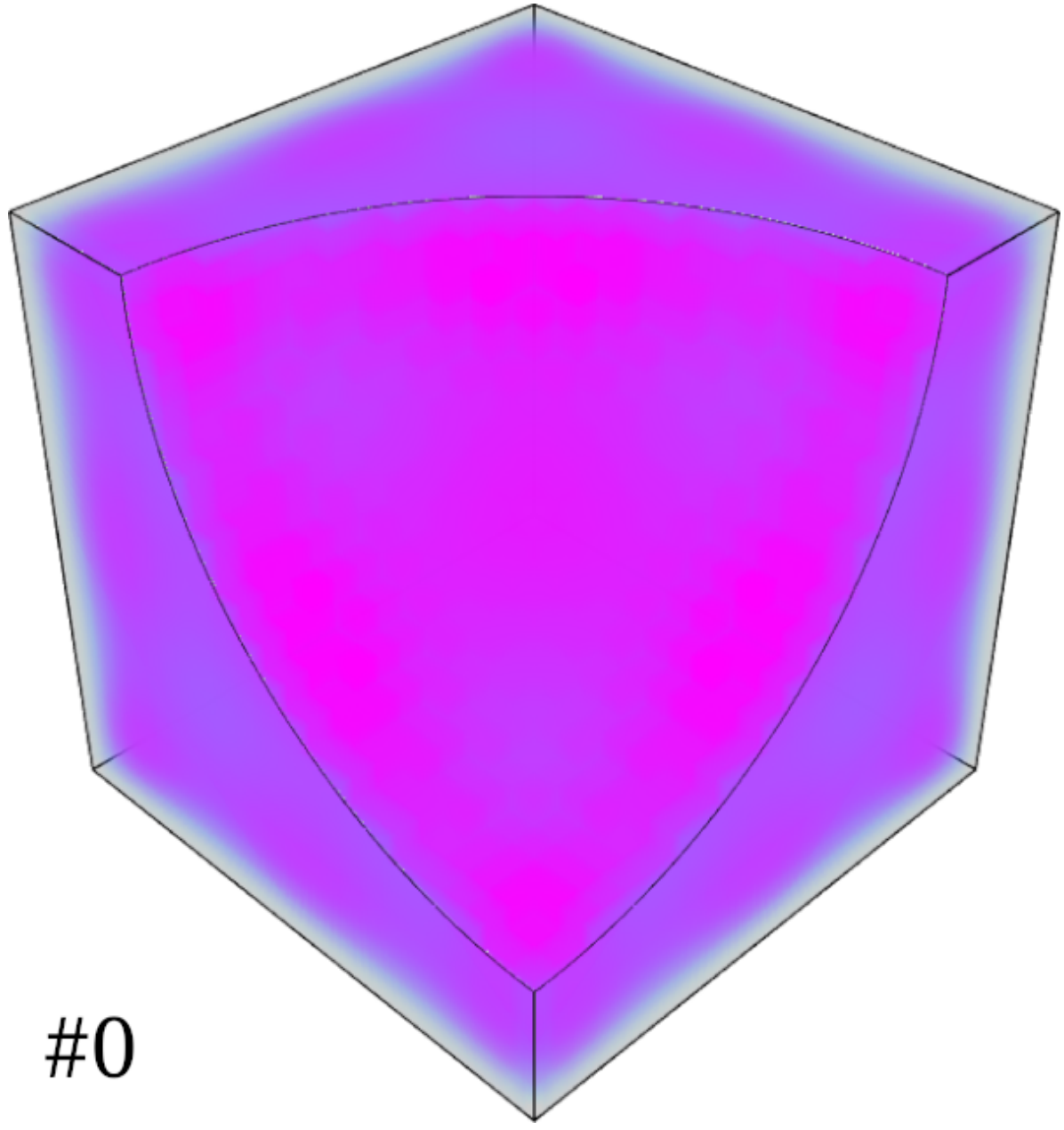


Quadrature toolbox: primal-dual RB

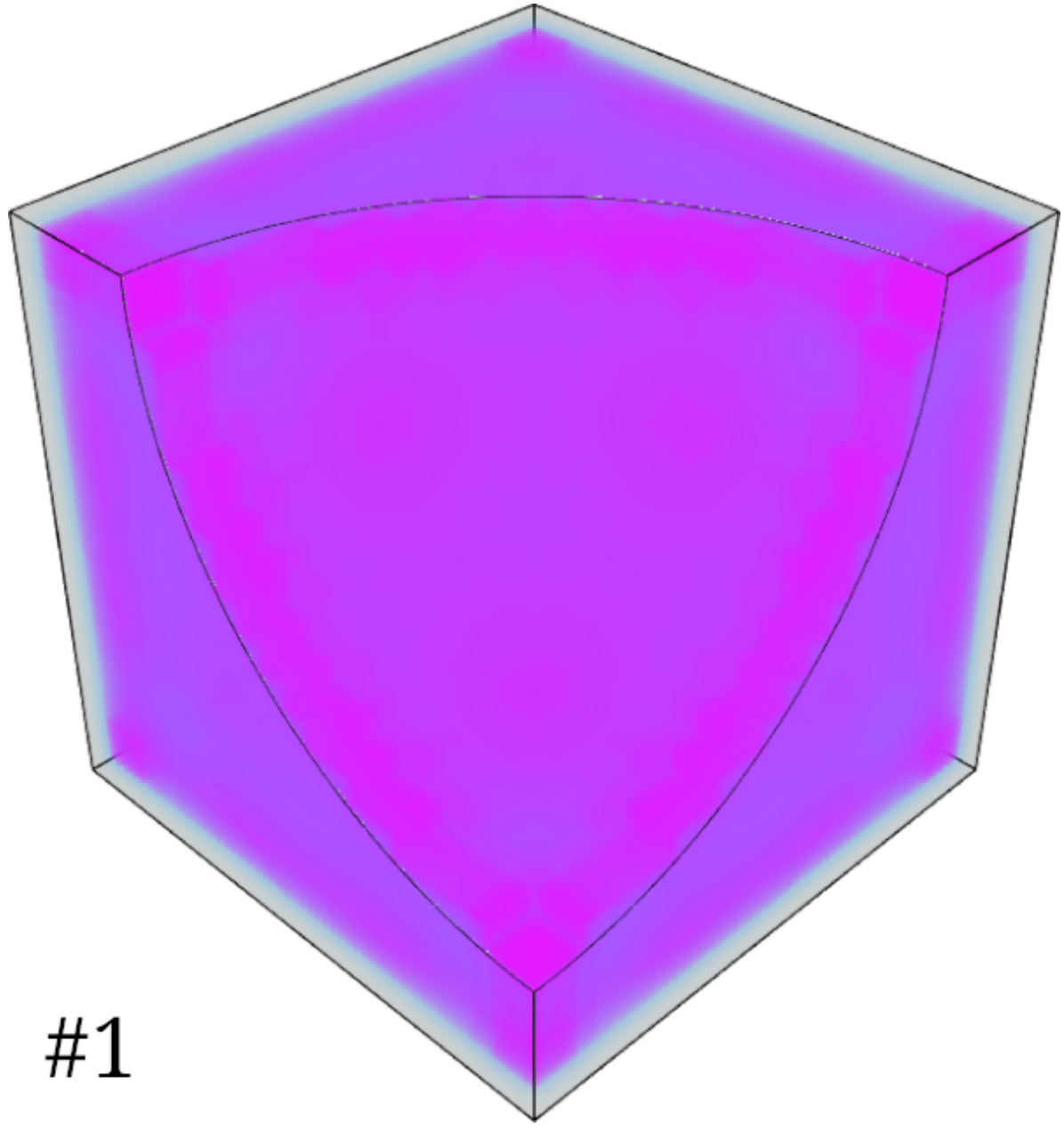
It works! And after only a single step. Just need to find local optima and re-optimize using Gauss-Newton after selection:



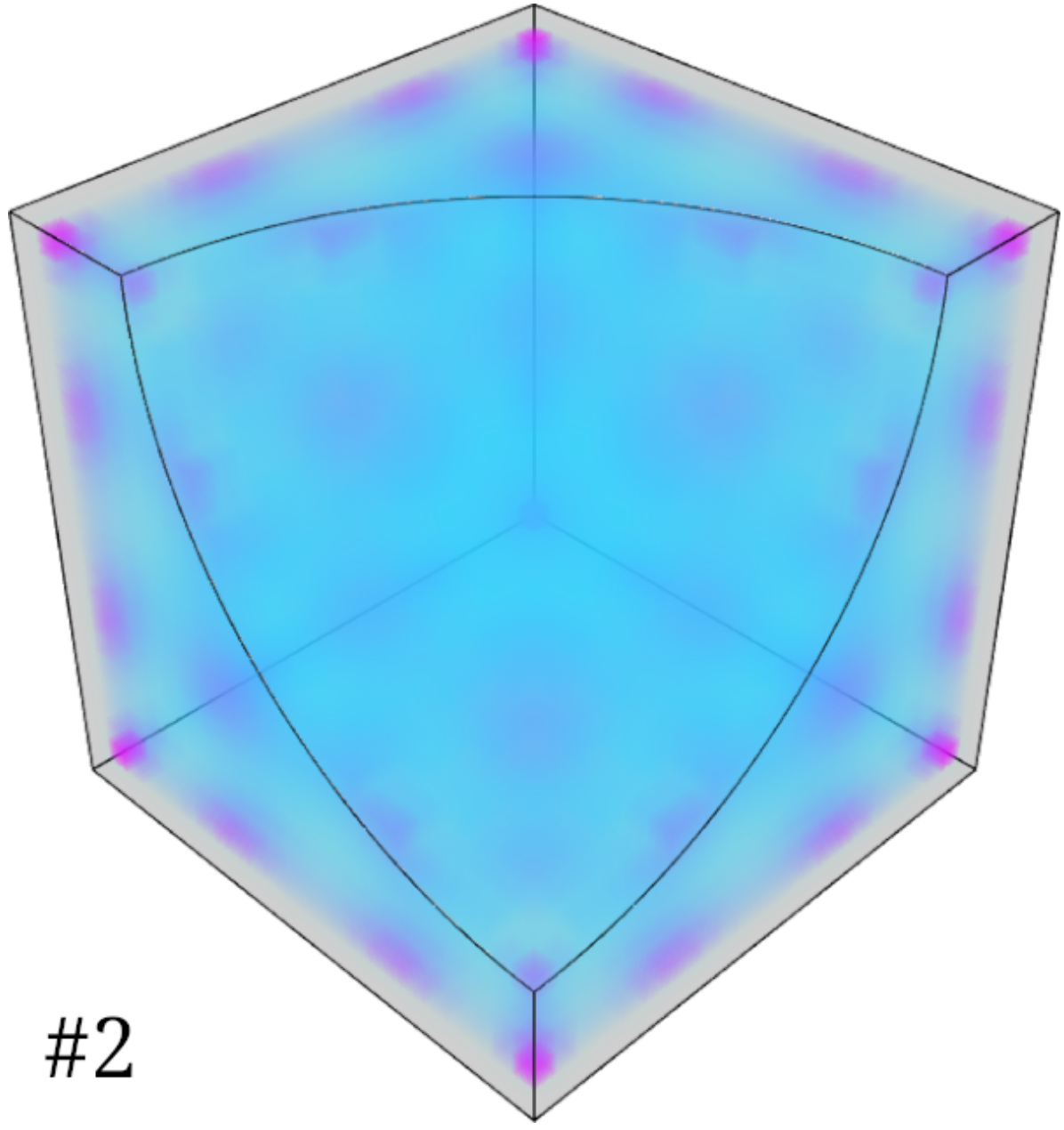
Quadrature toolbox: primal-dual RB



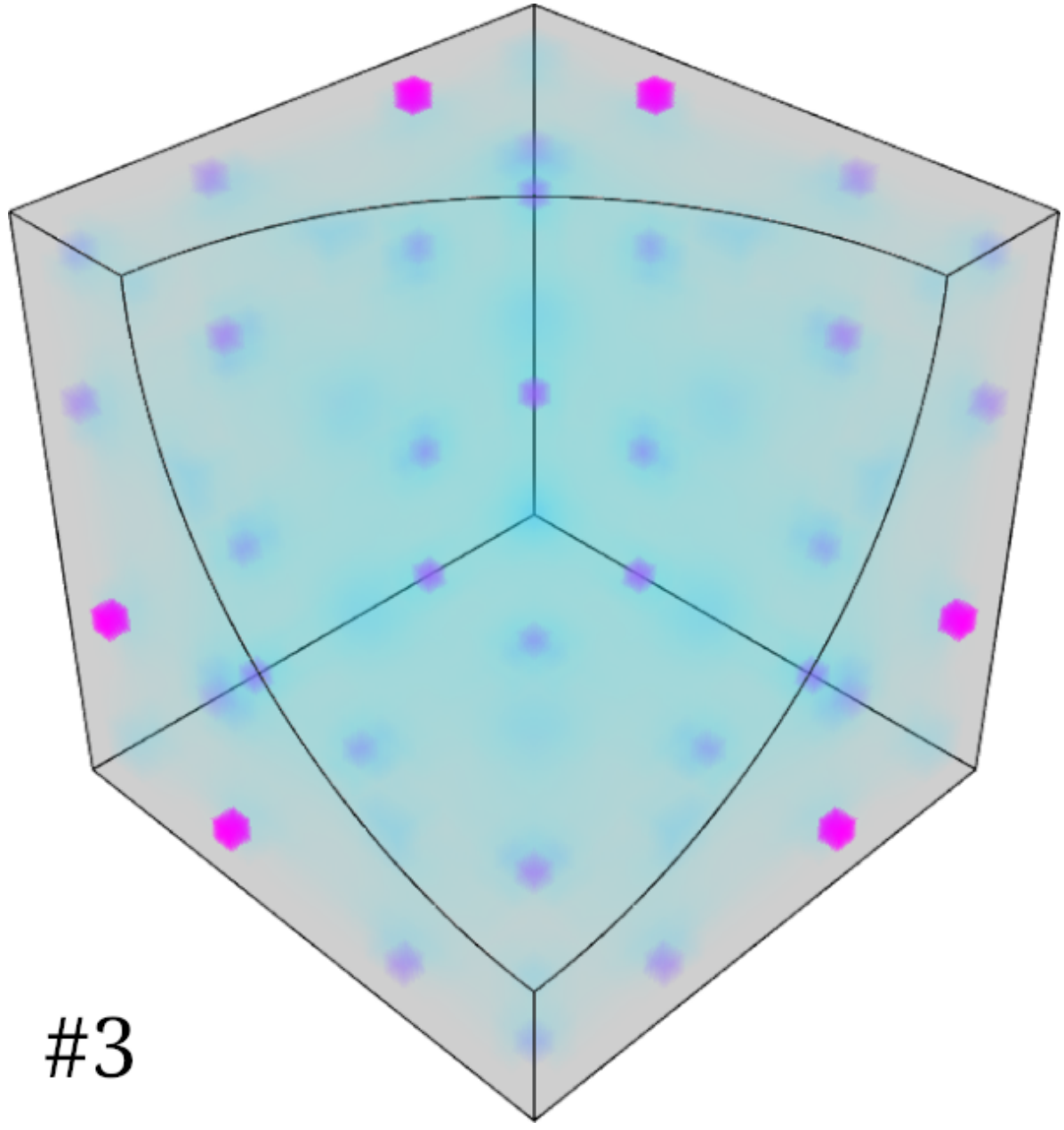
Quadrature toolbox: primal-dual RB



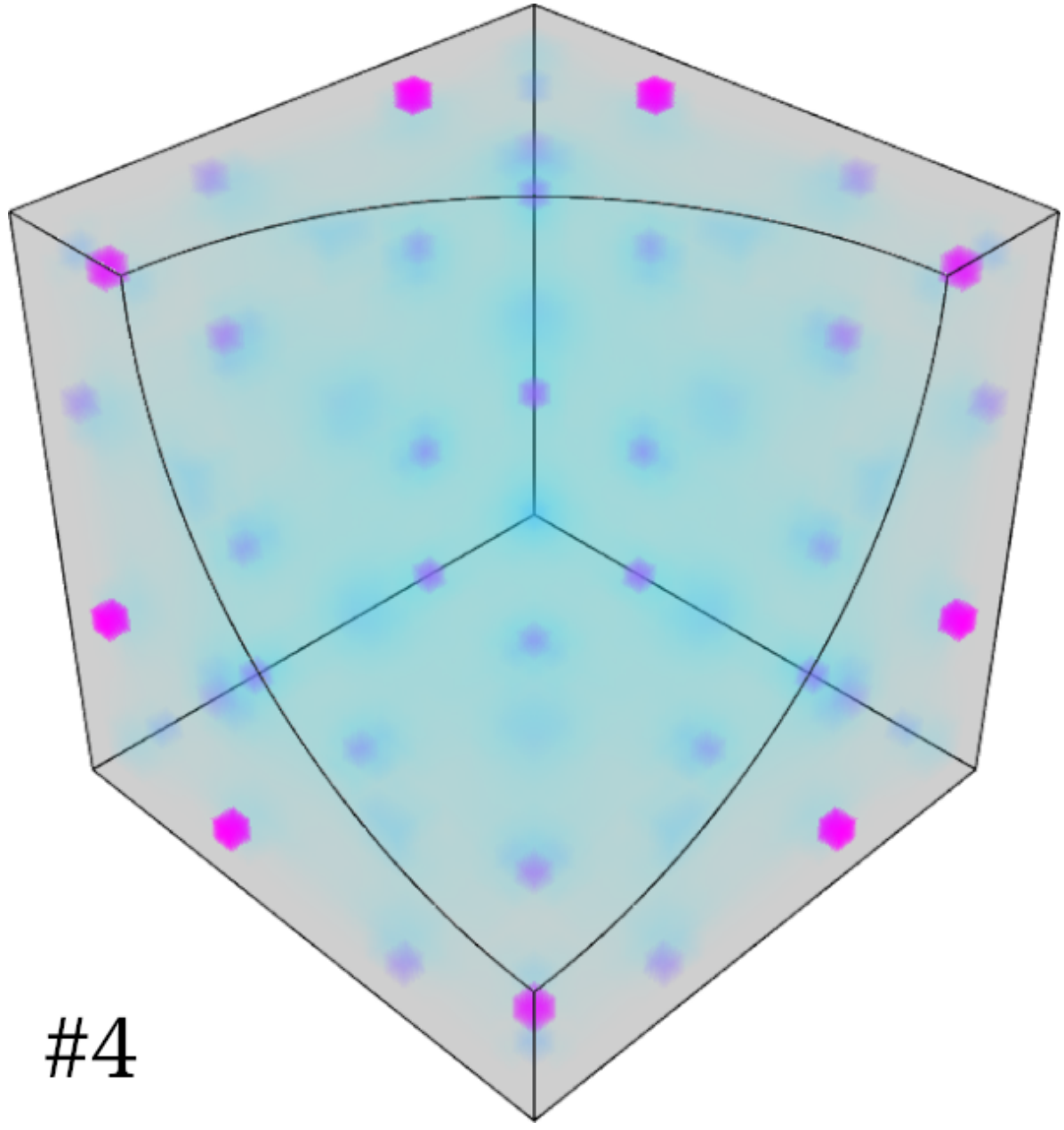
Quadrature toolbox: primal-dual RB



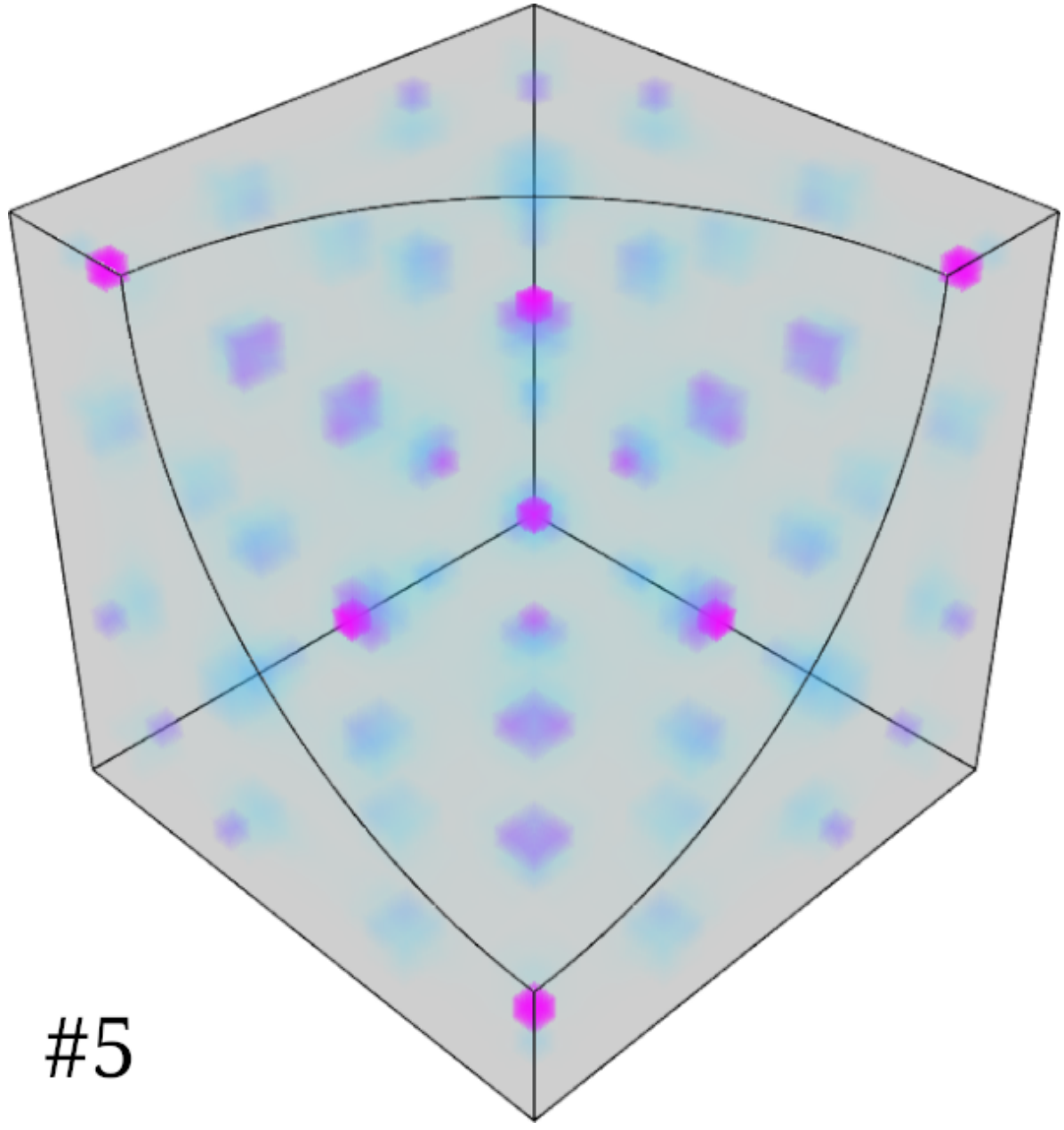
Quadrature toolbox: primal-dual RB



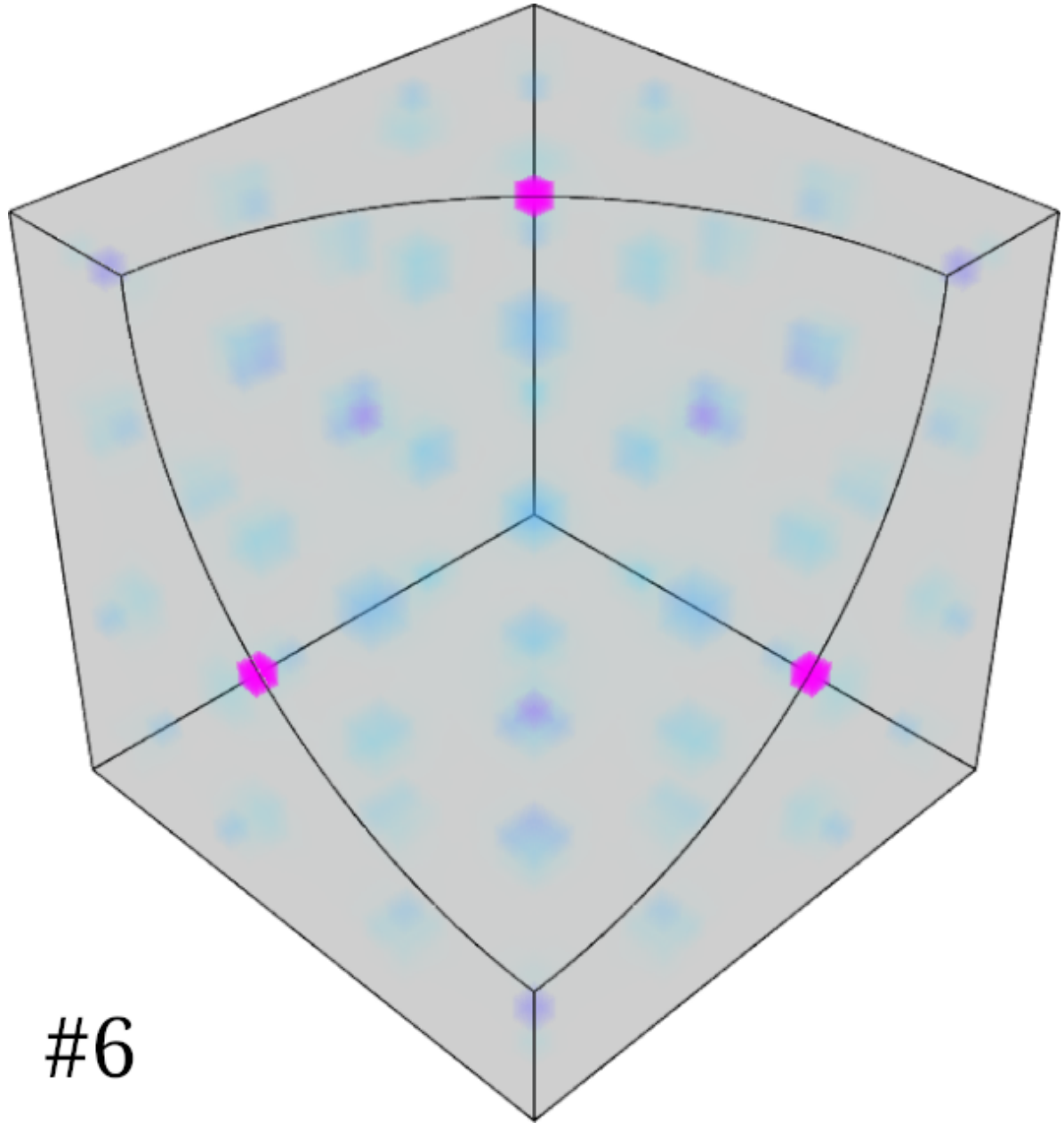
Quadrature toolbox: primal-dual RB



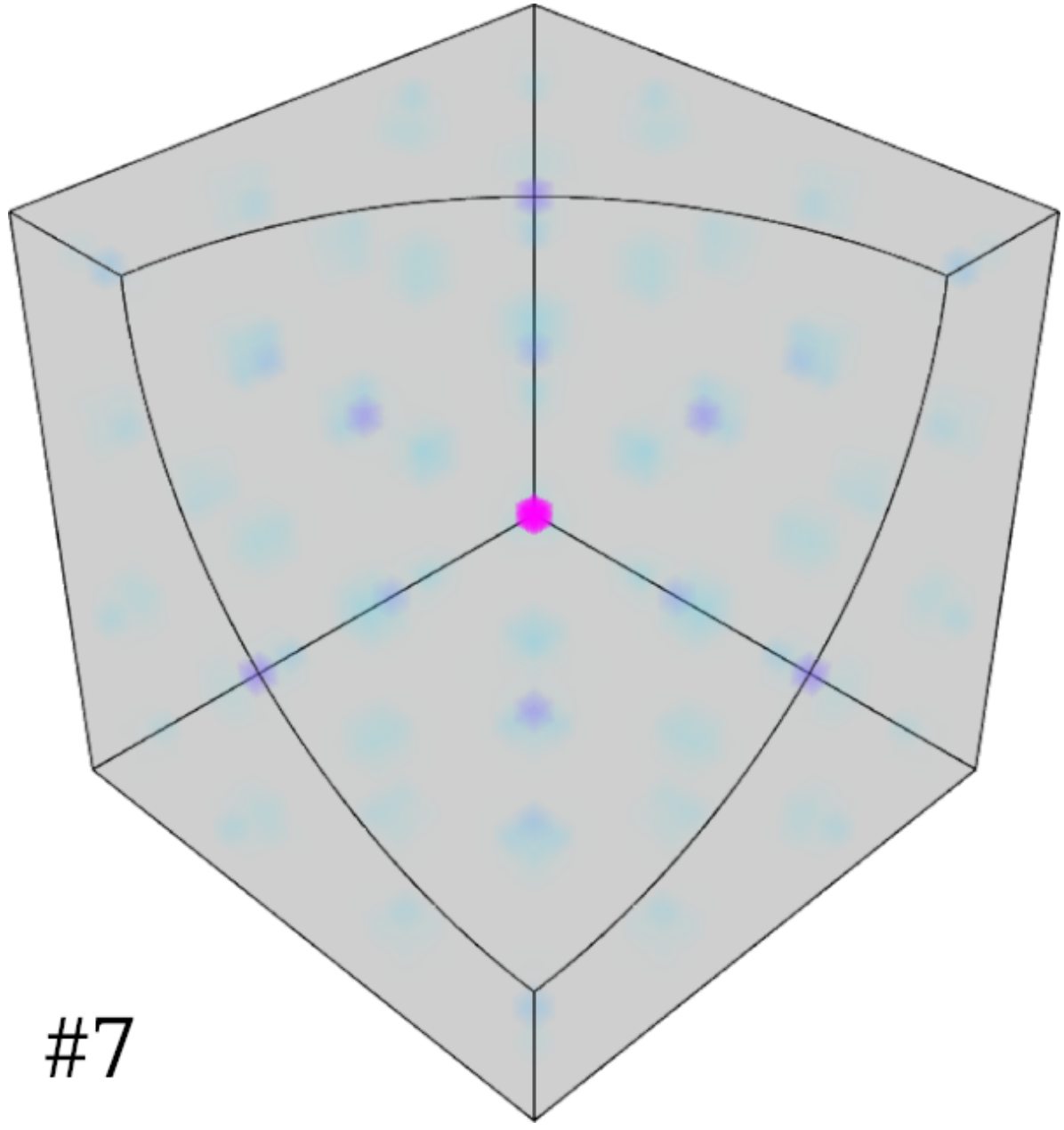
Quadrature toolbox: primal-dual RB



Quadrature toolbox: primal-dual RB

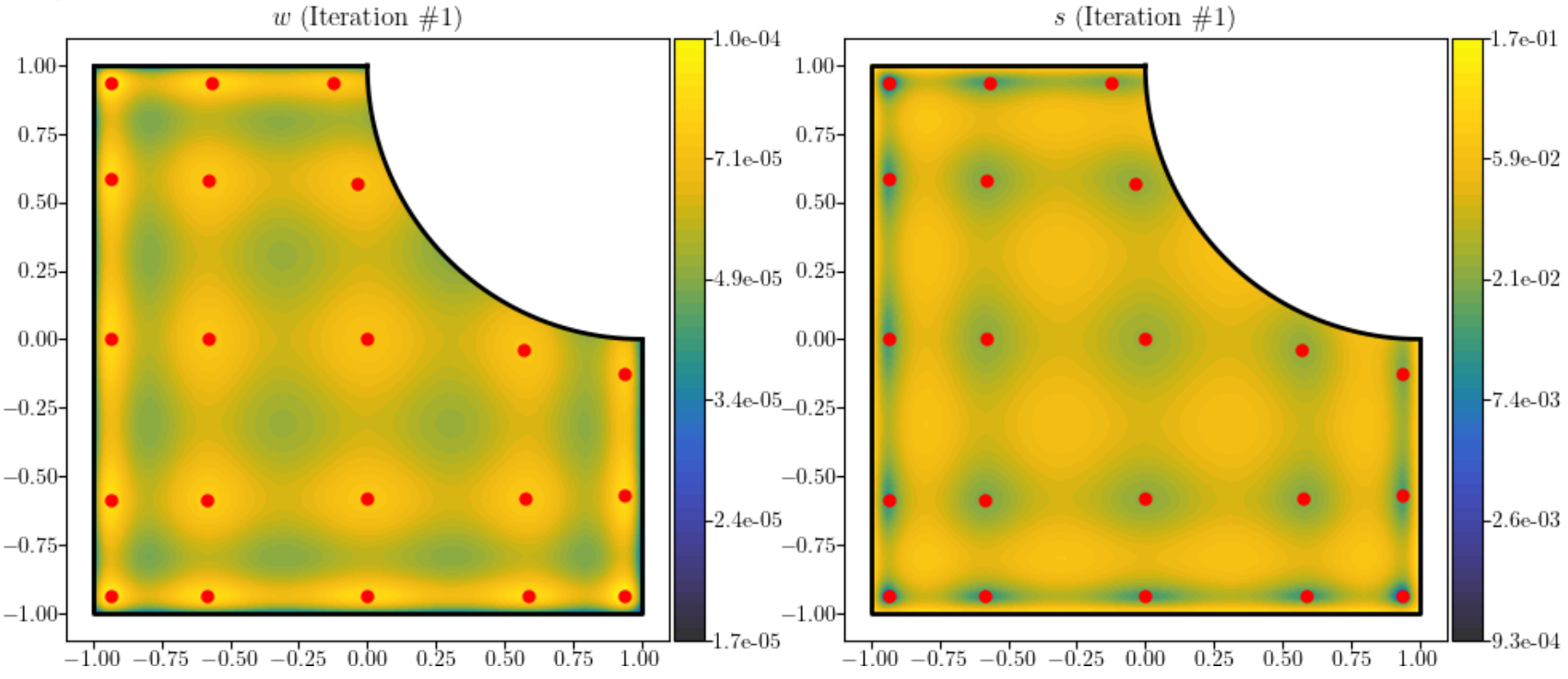


Quadrature toolbox: primal-dual RB



Quadrature toolbox: primal-dual RB

What about the dual variables? (Left: primal variable, right: dual variable)



Quadrature toolbox: dual asymptotic RB

What *are* the dual variables? Explicitly, to initialize the primal-dual iteration, we compute:

$$w^{(0)} \leftarrow V(V^{\mathsf{T}}V)^{-1}I \qquad \text{(initialize w/ Alg. LS)}$$
$$\lambda^{(0)} \leftarrow (V^{\mathsf{T}}V)^{-1}V^{\mathsf{T}}r \qquad \text{(compute Lagrange mult.)}$$
$$s^{(0)} \leftarrow r - V\lambda^{(0)} \qquad \text{(compute slack var.)}$$

Observations:

- The Lagrange multiplier $\lambda^{(0)}$ gives the coefficients in the ϕ_α basis of the projection of r onto $\mathbb{P}_{m,d}$ with respect to the uniform discrete measure supported on our sampling grid.
- Hence, the slack variable $s^{(0)}$ is the grid function which is the vector rejection of r—i.e., the part of r not in $\mathbb{P}_{m,d}$ with respect to the discrete measure.

Quadrature toolbox: dual asymptotic RB

Does this make sense? Yes. On the interval $[-1, 1]$ and with respect to the L^2 inner product, if we "vector reject" the monomial x^{2n} from $1, x, \ldots, x^{2n-1}$, we get the Legendre polynomial P_{2n}. Furthermore, the locations of the local minima of P_{2n} are asymptotic to the zeros of P_n (i.e., the order n Gauss-Legendre abscissae) (see e.g. Szëgo).

Some evidence: on the interval $[-1, 1]$, Gauss-Newton appears to always converge starting from the local minima of P_{2n}.

Fast primal/dual algorithms

Fast primal Ryu-Boyd:

- Run primal-dual interior point method for one or two steps.
- Find local maxima of w.
- Pull out those nodes; solve for weights using LS or NNLS.
- Optimize result w/ Gauss-Newton.

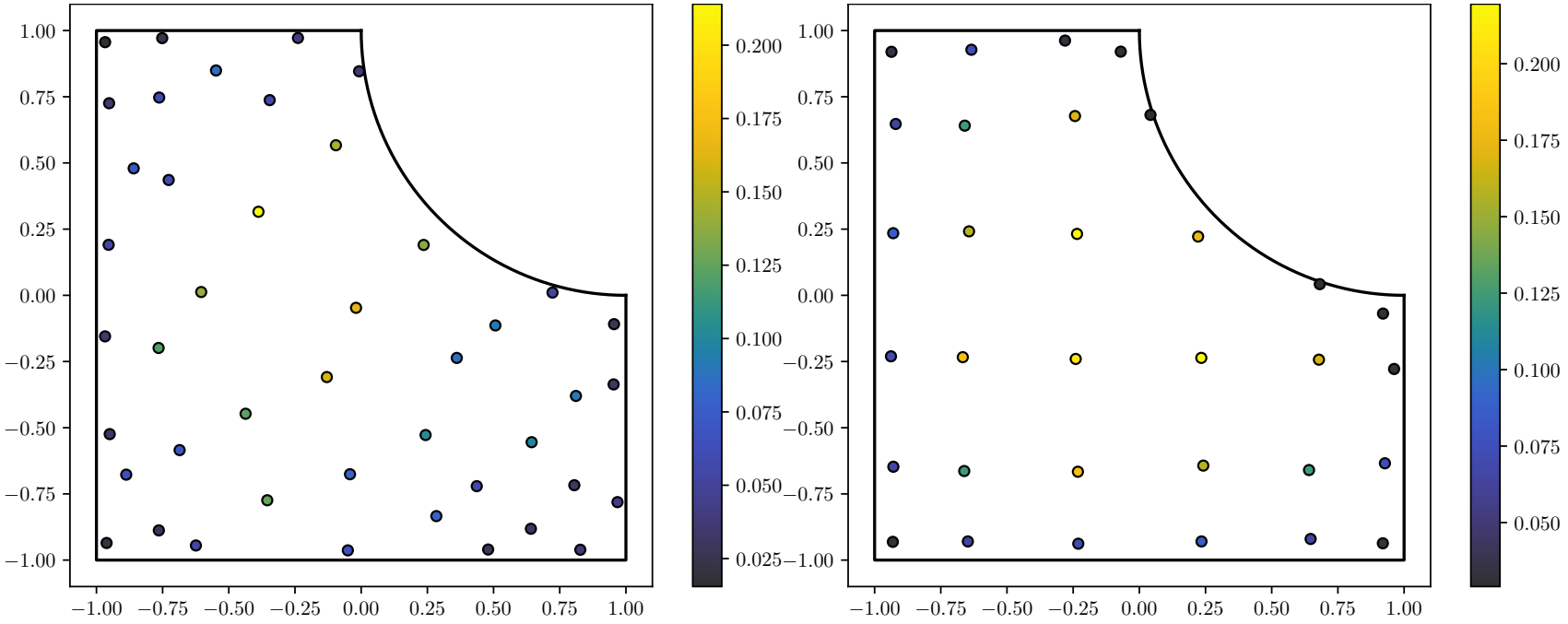
Fast dual Ryu-Boyd:

- Compute $s = r - V(V^\mathsf{T}V)^{-1}V^\mathsf{T}r$.
- Find local minima of s.
- Same deal as before.

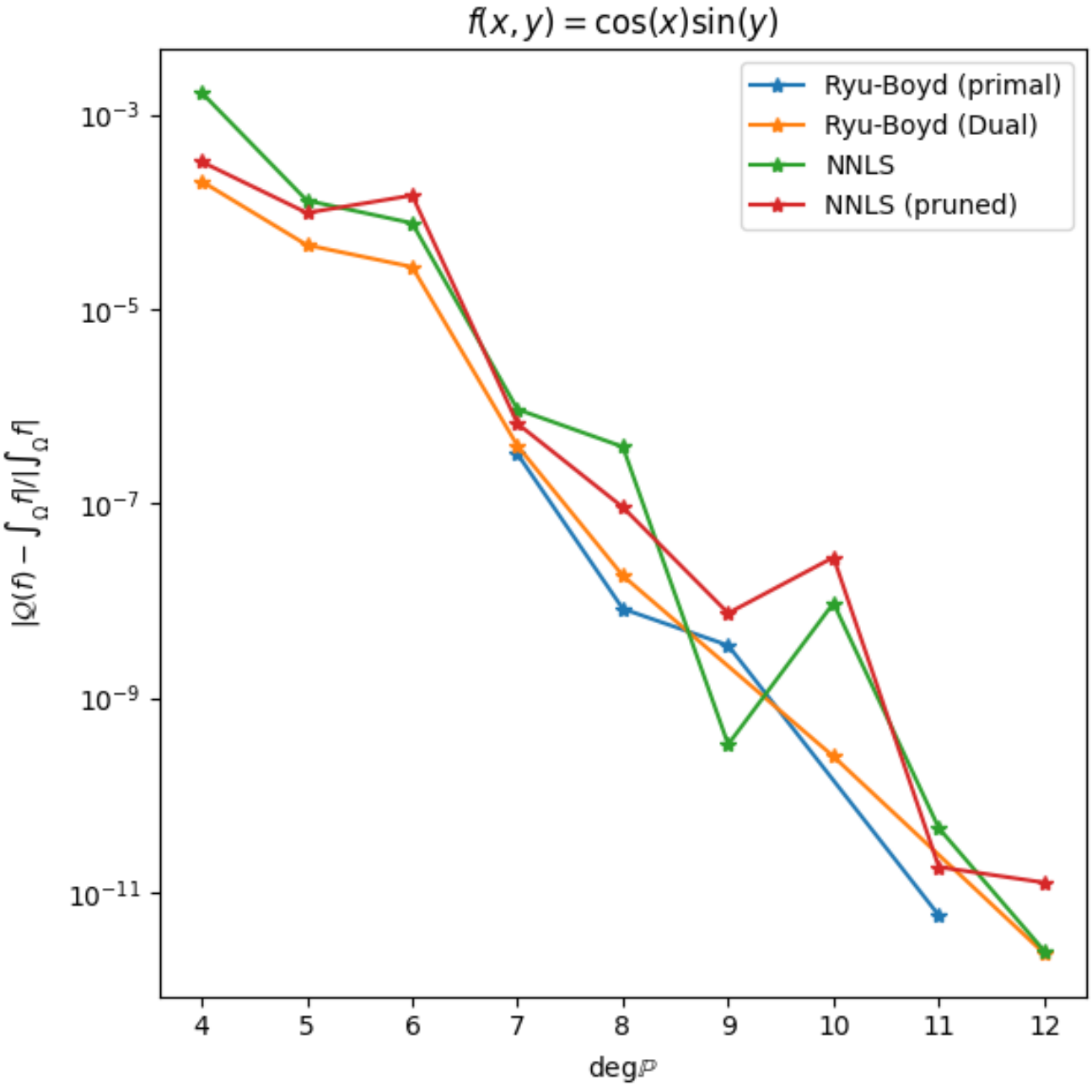
Both of these methods are heuristic!

Numerical results

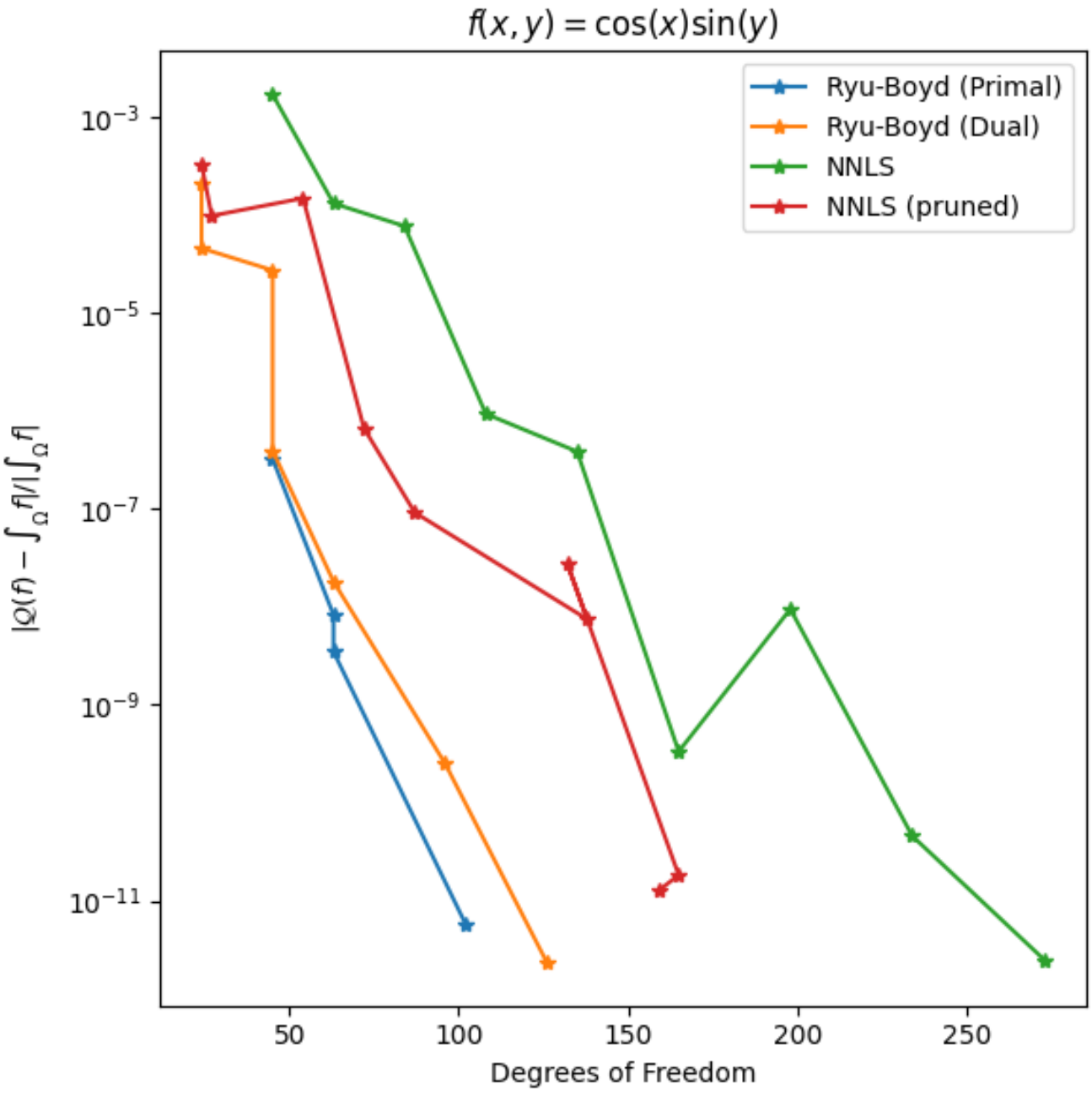
Vary degree, integrate different functions on one of our test domains, compare DOFs, accuracy, time to build rule



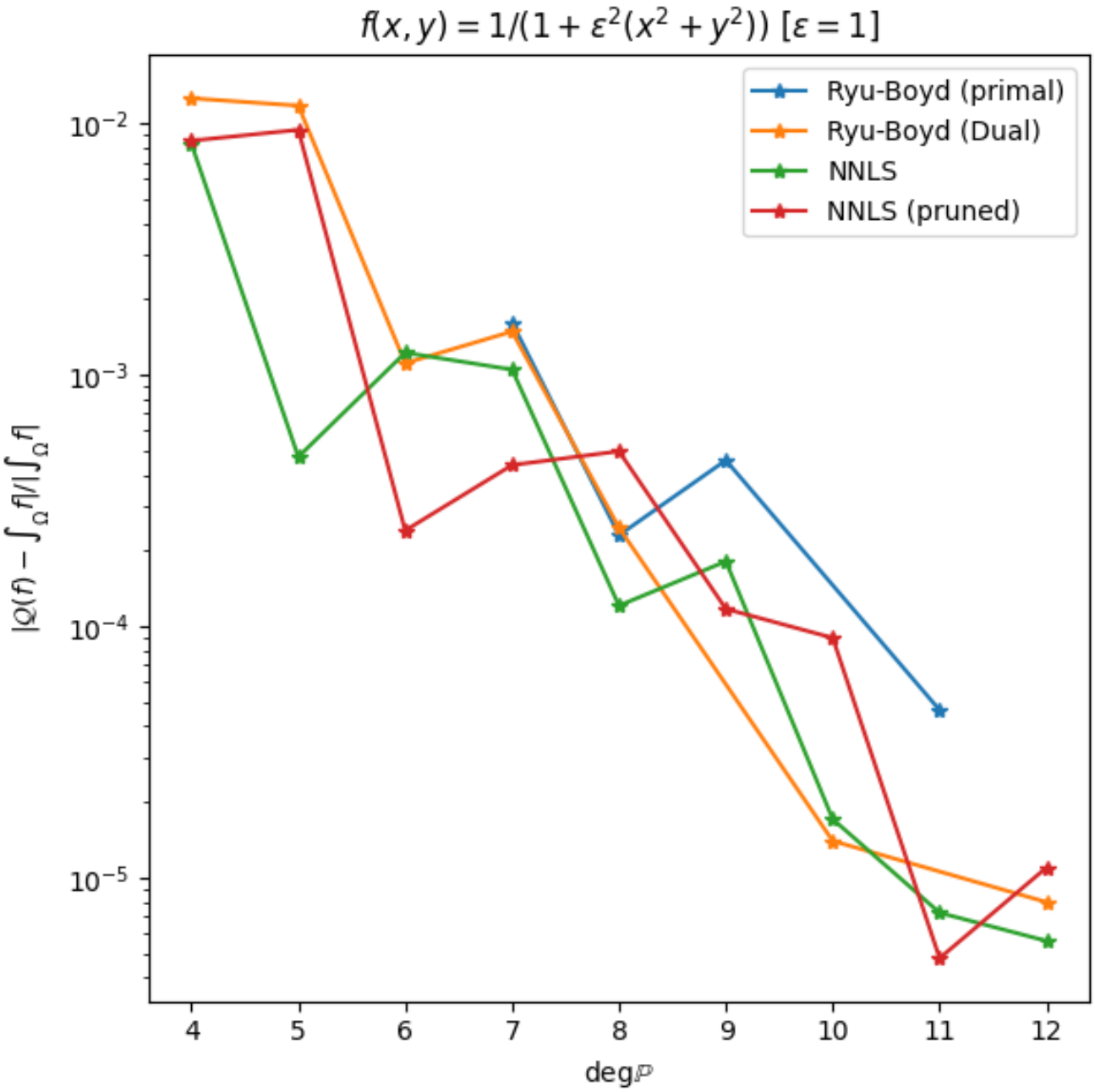
Numerical results



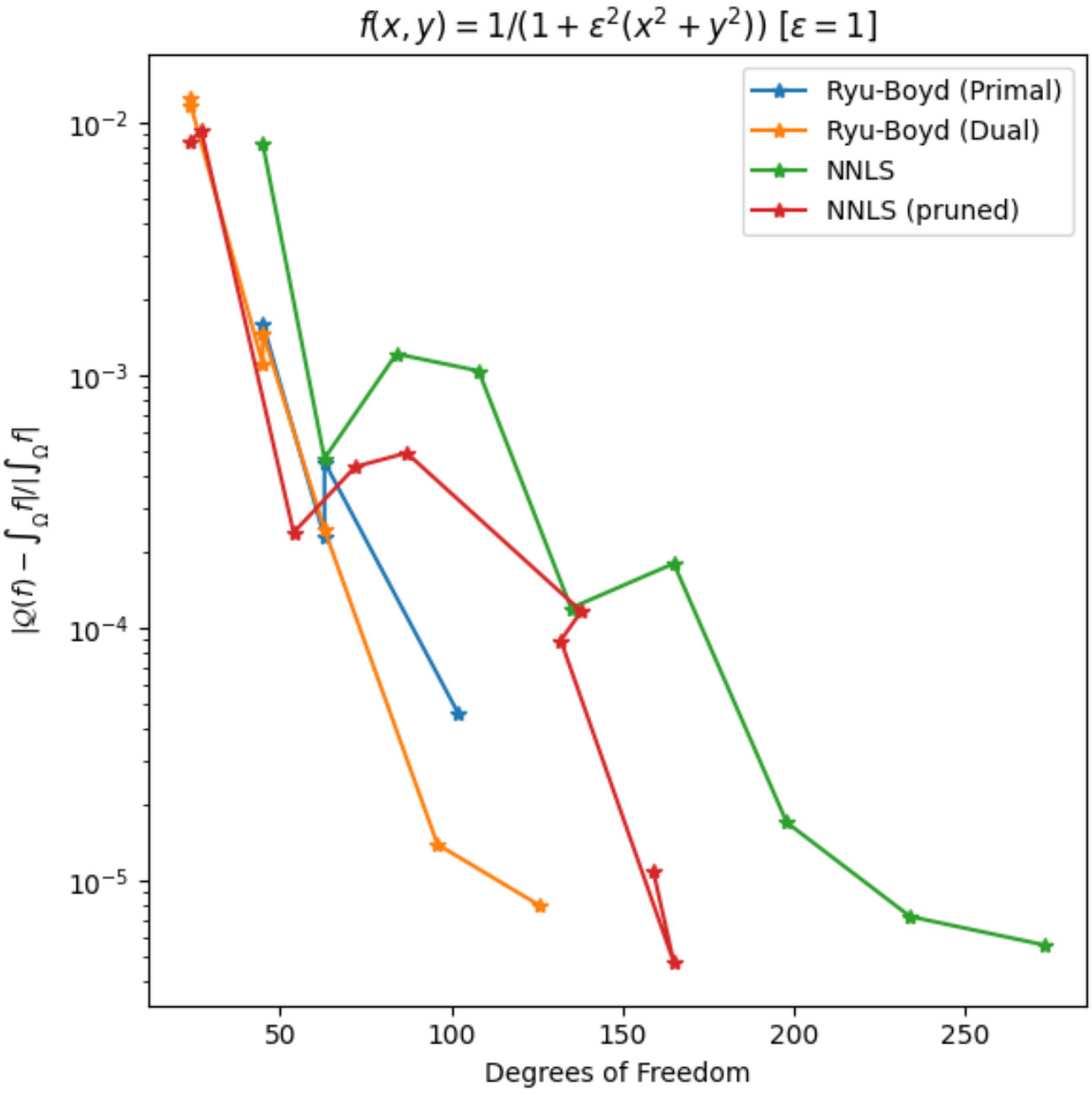
Numerical results



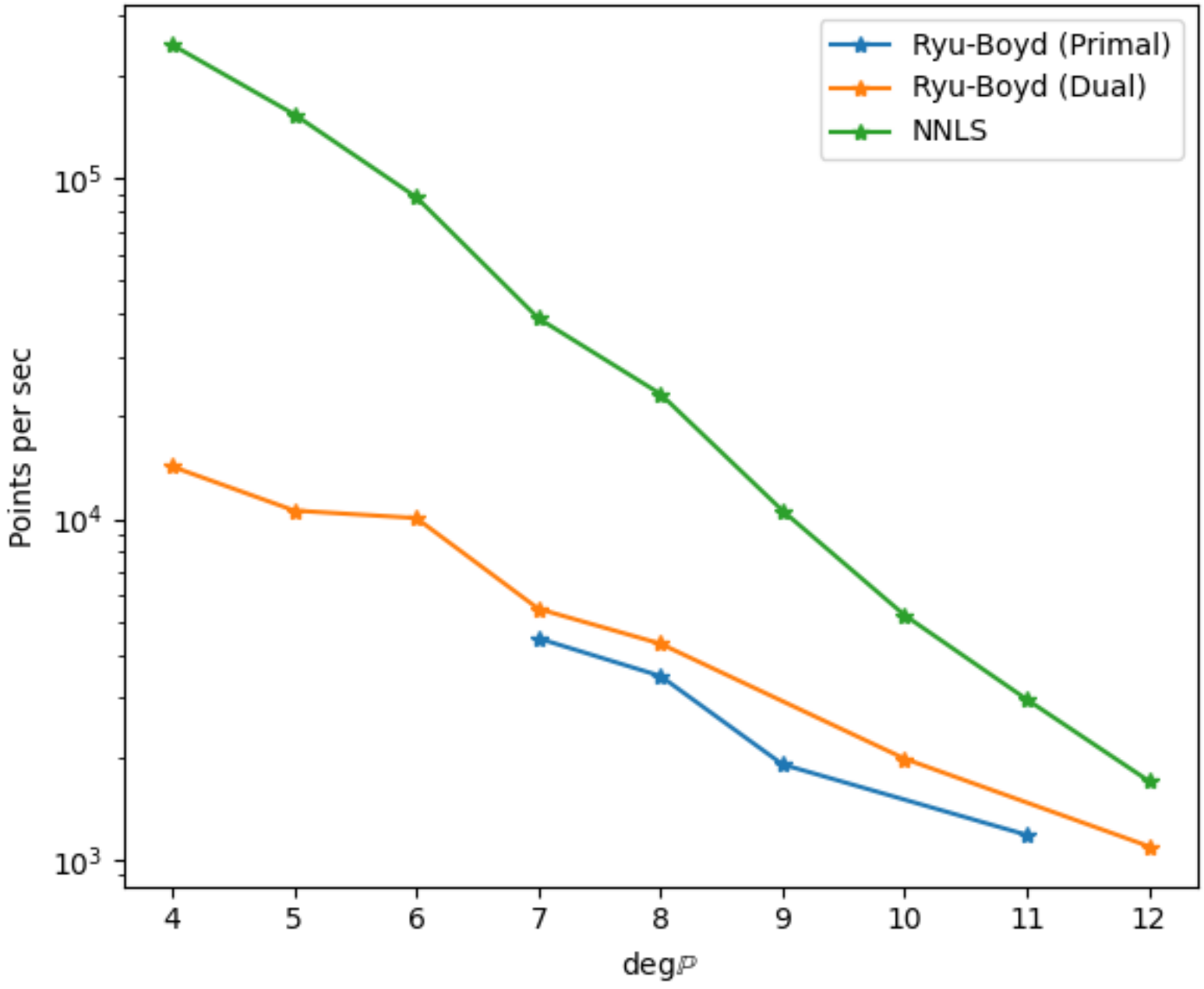
Numerical results



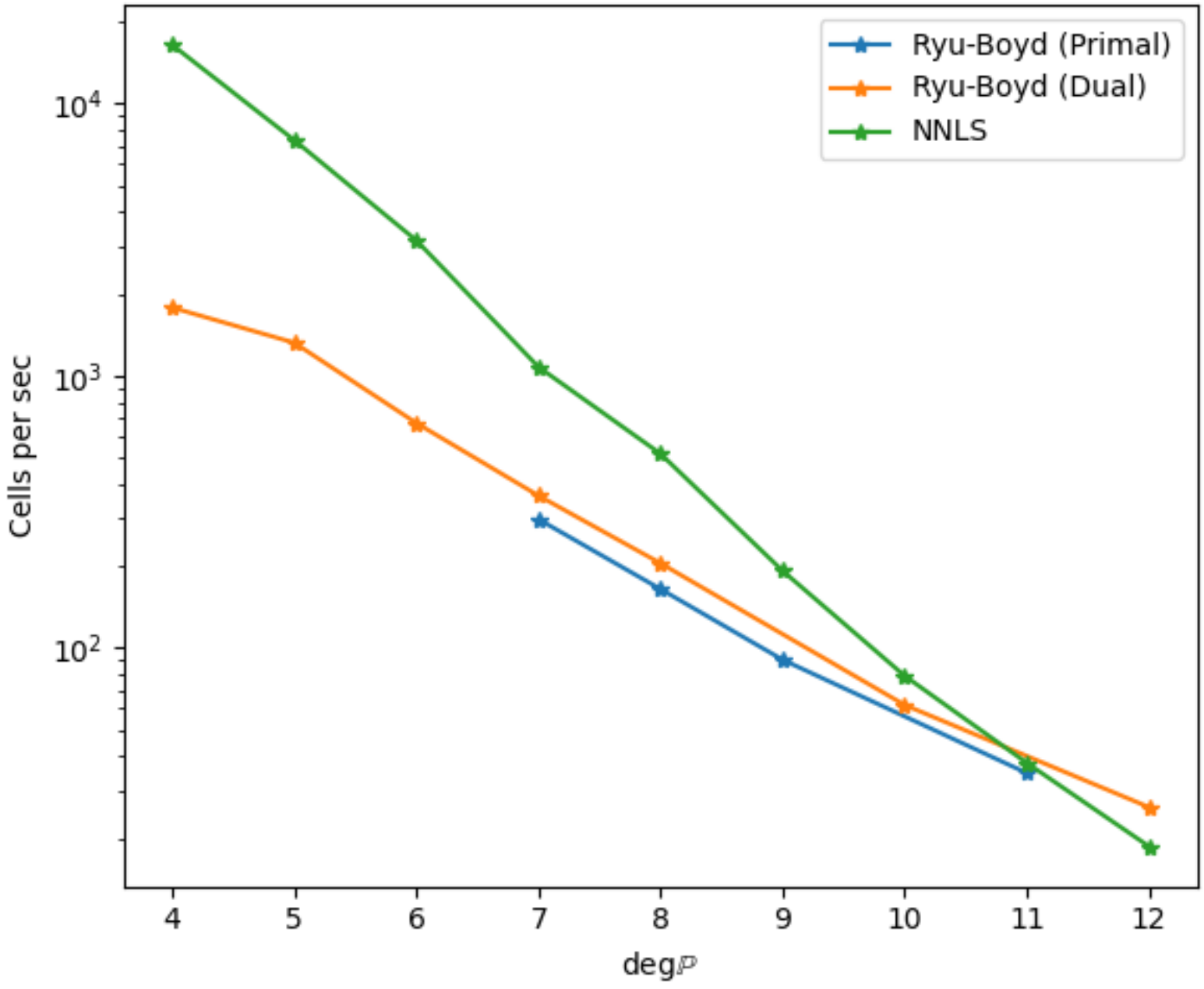
Numerical results



Numerical results

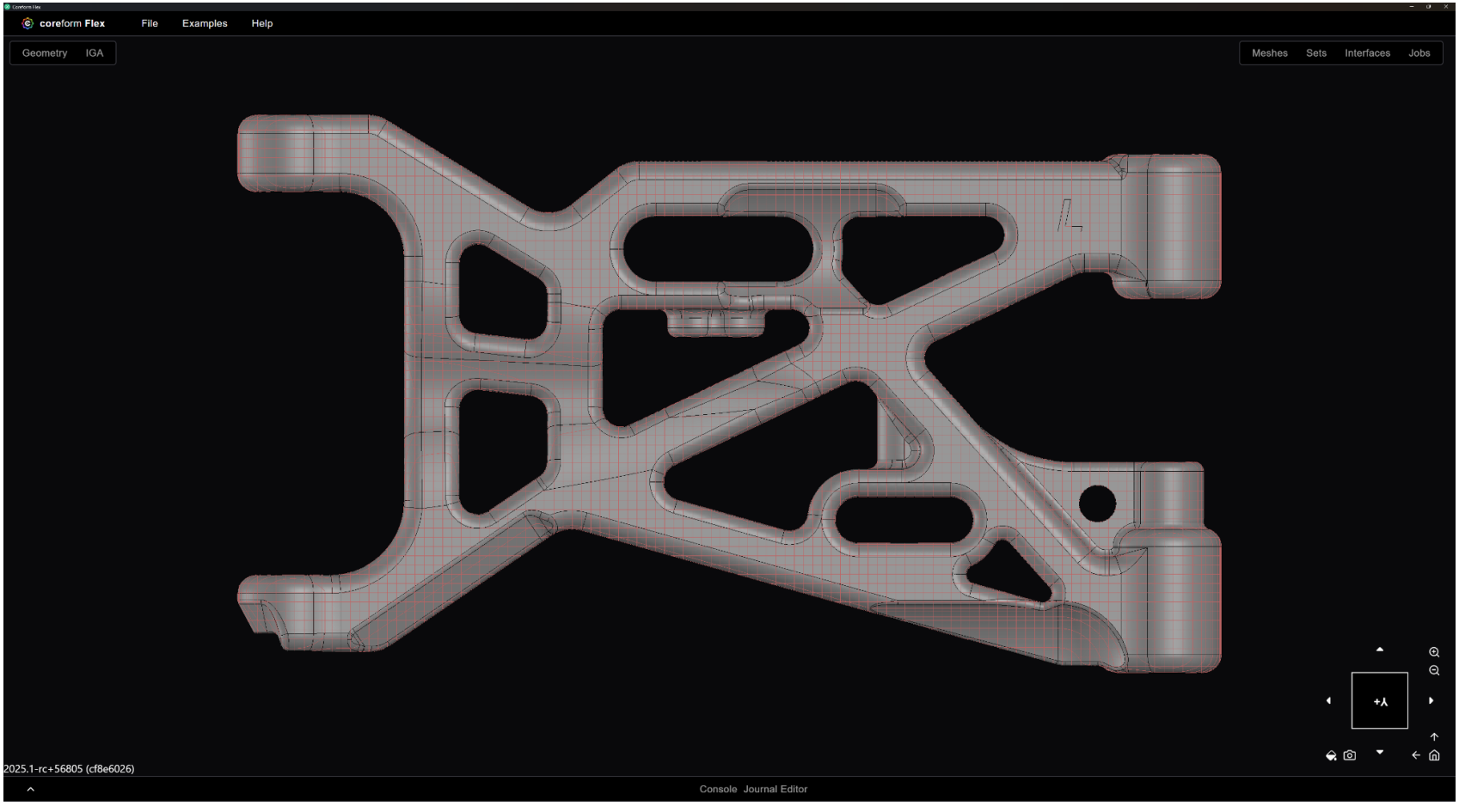


Numerical results

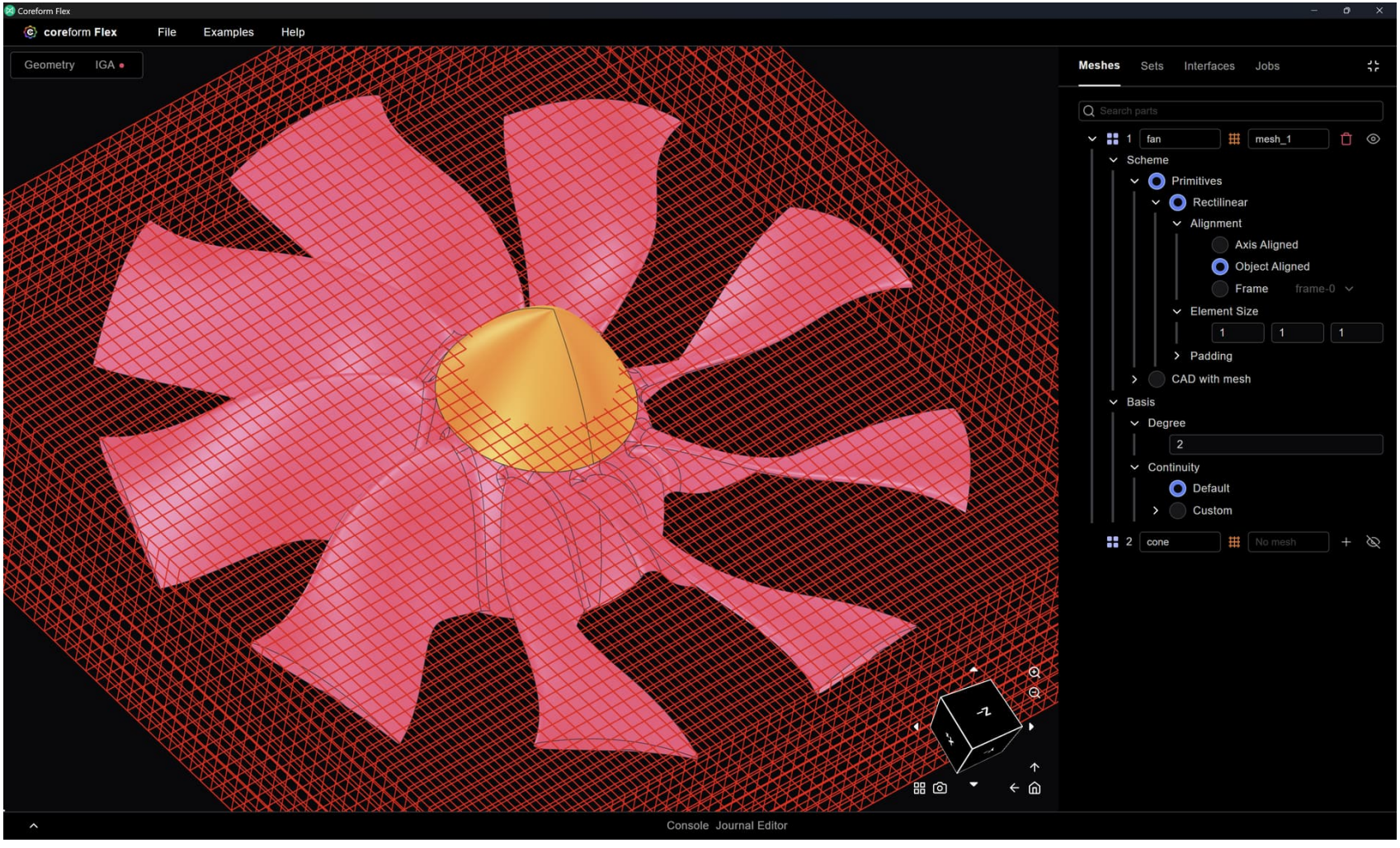


Use in Coreform Flex

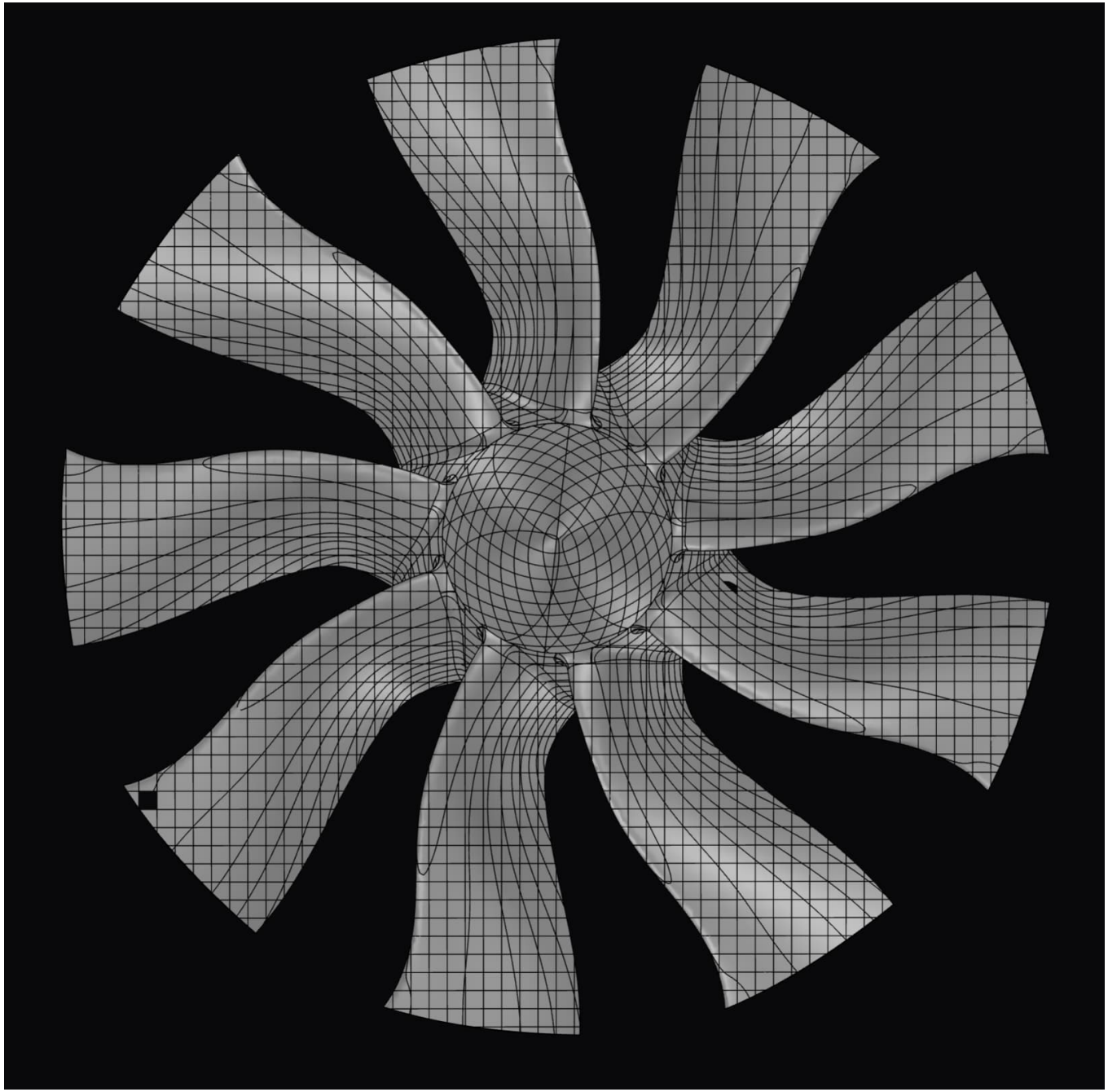
10x runtime speedup, 14x memory reduction



Use in Coreform Flex



Use in Coreform Flex



Takeaway

Thoughts:

- **Key takeway** is that there are many options to compute excellent quadrature rules on unstructured domains, allowing us to avoid mapping, meshing, decomposition, whatever.
- Lots of interesting things to try and to prove going forward to put this on firmer footing.
- The **most expensive** part of this process is setting up the moment vector I. Doing this quickly and accurately for a 3D simulation is “simple” but has a nontrivial implementation.

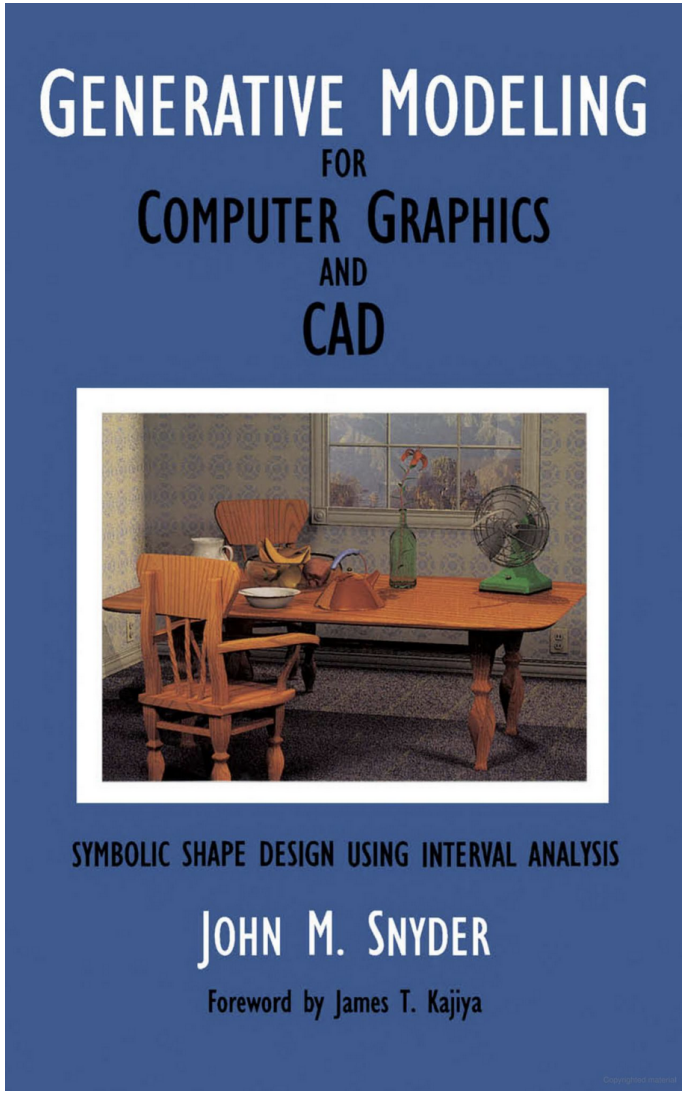
Paper:

- “Fast construction of efficient cut cell quadratures”, Samuel F. Potter, link to draft available on my website https://sampotter.github.io.

Robust contouring algorithms

Credit where credit's due





Contouring problems

Goal: solve nonlinear systems **robustly**

Example: for $\phi : \mathbb{R}^2 \to \mathbb{R}$, find all (x, y) such that $\phi(x, y) = 0$; i.e., find 2D zero level set of ϕ.

The beast: given $F : \mathbb{R}^n \to \mathbb{R}^m$ where $m < n$, find all $x \in \Omega \subseteq \mathbb{R}^n$ such that $F(x) = 0$.

Can usually continue along direction in nullspace of (full rank) D$F(x)$ and maintain $F(x) = 0$.

So, solution set is (generically) dimension $n - m$.

Note: $n = m$ is a "rootfinding" problem

Contouring problems

More examples: $F : \mathbb{R}^2 \to \mathbb{R}^1$:

- General level set/SDF contouring: $\phi(u, v) = 0$
- Sectioning: $\hat{n} \cdot (X(u, v) - x_0) = 0$
- 3D level set/SDF slicing: $\phi(x_0 + u\hat{t}_1 + v\hat{t}_2) = 0$
- 3D level set/surface intersection: $\phi(X(u, v)) = 0$
- Silhouette finding: $\hat{n}_X(u, v) \cdot (X(u, v) - x_{\text{eye}}) = 0$
- Diffeomorphism testing: Hadamard theorem says we need $\det \mathrm{D}\Phi(u, v) \neq 0$ ($\Phi : \mathbb{R}^2 \to \mathbb{R}^2$)
- Surface developability: $EF - G^2 = 0$

Contouring problems

More examples: $F : \mathbb{R}^2 \to \mathbb{R}^{\mathbf{2}}$:

- Robust 2D global optimization: to solve $f(x, y) = 0$, contour $f_x = 0$ and $f_y = 0$ separately, intersect the curves to get set of all extrema
- Curve-curve Hausdorff distance: $d(s, t) = \|x(s) - y(t)\|_2^2$, contour $d_s = 0$ and $d_t = 0$, find all saddle points in intersection of those two curves, ...

Contouring problems

More examples: $F : \mathbb{R}^3 \to \mathbb{R}^m$:

- Level set/SDF contouring ($m = 1$): $\phi(\vec{x}) = 0$
- Level set/SDF silhouette finding ($m = 2$): solve previous problem, intersect with $\nabla\phi(\vec{x}) \cdot (\vec{x} - \vec{x}_{\text{eye}}) = 0$
- "Global" filletting ($m = 1$): e.g., contour $-r$ SDF of surface, contour $+r$ SDF of the result
- Lots of other games to play with SDFs

Contouring problems

A few more:

- Curve-curve intersection ($m = 2$, $n = 2, 3$):
  $F(s,t) = x(s) - y(t)$
- Curve-surface intersection ($m = 1 + 2$, $n = 3$):
  $F(s,u,v) = x(s) - Y(u,v) = 0$
- Surface-surface intersection ($m = 2 + 2$, $n = 3$):
  $F(s,t,u,v) = X(s,t) - Y(u,v) = 0$
- Curve-surface and surface-surface Hausdorff distances

Lots unknown! Contour each component separately and intersect, or all together? I don't know! John's book has ~1 page about this.

Contouring problems

Four approaches to solving **underdetermined** nonlinear rootfinding problems (i.e., **contouring** problems):

- Algebraic methods
- Homotopy methods
- Continuation methods
- Subdivision methods

Briefly discuss each of these...

Algebraic methods

Just use algebra and solve the system... no big deal, right?

(Well, I guess we need F to be rational or a polynomial...)

Algebraic methods

Computation of the solutions of nonlinear polynomial systems

Evan C. Sherbrooke and Nicholas M. Patrikalakis

Department of Ocean Engineering, Design Laboratory, Massachusetts Institute of Technology, Cambridge, MA 02139-4307, USA

Received November 1991
Revised February 1993

Abstract

Sherbrooke, E.C. and N.M. Patrikalakis, Computation of the solutions of nonlinear polynomial systems, Computer Aided Geometric Design 10 (1993) 379–405.

Algebraic methods

The computation of all complex roots of nonlinear polynomial systems has typically been approached with *algebraic geometry techniques* like elimination or Gröbner basis methods [Canny '90, Buchberger '85]. These methods have many advantages; they are theoretically elegant, guaranteed to find all complex roots of a system irrespective of the dimensionality of the solution set and well-suited for implementation in symbolic mathematical systems. However, they suffer from numerical instability, making implementation in floating point arithmetic difficult. Furthermore, they are inefficient in memory and processing time requirements, making them unattractive for systems only moderate in degree or dimensionality. There is also a philosophical problem with these techniques: they frequently give us much more information than we need. As we shall see in Section 7, many CAD applications only require those roots which lie in some real n-dimensional box.

(From Sherbrooke & Patrikalakis)

Algebraic methods

Genus of the intersection curve of two rational surface patches

Sheldon KATZ

Oklahoma State University, Stillwater, OK 74078, U.S.A.

Thomas W. SEDERBERG

Brigham Young University, Provo, UT 84602, U.S.A.

Received January 1988
Revised April 1988

Abstract. It is shown that two generic triangular surface patches with no base points and of parametric degree m and n respectively, intersect in a curve of degree m^2n^2 which is generally of genus $2m^2n^2 - \frac{3}{2}m^2n - \frac{3}{2}n^2m + 1$. Similarly, two generic tensor product surface patches of parametric degree $m_1 \times m_2$ and $n_1 \times n_2$ respectively, intersect in a curve of degree $4m_1m_2n_1n_2$ and generally of genus $8m_1m_2n_1n_2 - 2m_1m_2(n_1 + n_2) - 2n_1n_2(m_1 + m_2) + 1$. For example, two general bicubic patches in general position intersect in a curve of degree 324 and of genus 433. The significance of this genus value lies in the fact that only curves of genus 0 can be expressed parametrically using rational polynomials.

Genus and degree equations are also derived for intersection curves involving surface patches with simple base points. A class of surfaces is identified for which any plane section is a rational curve.

Algebraic methods

Numerical Instability of Algebraic Rootfinding Methods*

Emil Graf and Alex Townsend

Abstract. We demonstrate that the most popular variants of all common algebraic multidimensional rootfinding algorithms are unstable by analyzing the conditioning of subproblems that are constructed at intermediate steps. In particular, we give multidimensional polynomial systems for which the conditioning of a subproblem can be worse than the conditioning of the original problem by a factor that grows exponentially with the number of variables.

Key words. polynomial systems, numerical instability, rootfinding

MSC codes. 13P15, 65H04, 65F35

1. Introduction. For over a decade, the scientific community has been searching for a numerically robust multivariate polynomial rootfinder [5, 9, 11, 13, 15, 19, 20, 23, 24]. We are yet to find one. This paper demonstrates that the most popular variants of all common algebraic multidimensional rootfinding algorithms are exponentially unstable. This includes hidden-variable resultants (see [15]), Gröbner bases (see section 3), the rational univariate representation (see section 4), multiparameter eigenvalue problems (see section 5), normal form methods (see section 6), and Macaulay resultants (see section 7). We are stuck waiting for new ideas to emerge from algebraic geometry.

Homotopy methods

Introduce auxiliary problem which is easy to solve, $F_0(x) = 0$, but whose solution set is of same type as original rootfinding problem.

Let x_0 be its solution.

Consider parametrized family of problems: $F(x,t) = (1-t)F_0(x) + tF(x) = 0$. Clearly, $F(x_0, 0) = 0$, and $F(x, 1) = F(x) = 0$ is solution to original problem.

Sweep t from 0 to 1 to construct a solution dependent on t (a homotopy) between $x_0 = x(0)$ and $x = x(1)$.

Homotopy methods

Computation of the solutions of nonlinear polynomial systems

Evan C. Sherbrooke and Nicholas M. Patrikalakis

Department of Ocean Engineering, Design Laboratory, Massachusetts Institute of Technology, Cambridge, MA 02139-4307, USA

Received November 1991
Revised February 1993

Abstract

Sherbrooke, E.C. and N.M. Patrikalakis, Computation of the solutions of nonlinear polynomial systems, Computer Aided Geometric Design 10 (1993) 379–405.

Homotopy methods

The second category of methods is the class of *homotopy techniques* [Zangwill & Garcia '81, Garcia & Zangwill '80]. These methods may be used to find all complex solutions of a nonlinear polynomial system if the number of roots is finite. Unfortunately, investigation of such methods indicates that they also tend to be numerically ill-conditioned. If we try to get around this problem by implementing the algorithm in exact rational arithmetic, we end up with enormous memory requirements because we have to solve large systems of complex initial value problems. And homotopy suffers from the same philosophical problem as algebraic geometry techniques—it simply provides more information than we need.

(From Sherbrooke & Patrikalakis)

Continuation methods

Simple idea: find one point on the solution (e.g., one point on the intersection curve), and trace out a solution. Take a step in the tangent direction (if it exists!) or solve a differential equation (if it has a solution!).

This is how most intersectors works (including ours).

One challenge: not clear how to extend to a solution set which has dimension greater than 1. Good luck contouring a two-dimensional level set.

Continuation methods

Improved test for closed loops in surface intersections

T W Sederberg, H N Christiansen and S Katz*

The following two theorems are proven: If two nonsingular curve segments intersect twice, if the range of normal vectors of each segment does not vary more than 90°, and if each curve is C^1 smooth, then there exists a line which is perpendicular to both curve segments simultaneously. If two nonsingular surface patches intersect in a closed loop, if the dot product between any normal vector on one surface and any other normal vector on either surface is never zero, and if the normal vector is uniquely defined at every point on each surface region, then there exists a line which is perpendicular to both surface patches simultaneously.

The second theorem is of value because it provides a criterion for eliminating a robustness limitation that arises in computing surface intersections using the marching method; namely, assuring that all branches of

a point within every closed loop in the intersection curve. Note that here a 'closed' loop is a loop whose image in the parameter space of *each patch* is closed (i.e. does not intersect any patch boundary curve).

Previous work on loop detection includes the following. Sederberg and Meyers[7] showed that if two surfaces (each C^1) intersect in a closed loop, then there exists a normal vector on one surface that is perpendicular to a tangent vector on the other surface. Sinha *et al.*[8] proves that if two surfaces (each C^1) intersect in a closed loop, then there exists a normal vector on one surface that is parallel to a normal vector on the other surface.

In this paper it is shown that if two surface patches are each C^1 and if the dot product between any normal vector on one surface and any other normal vector on either surface is never zero, then there exists a line

(1989)

Continuation methods

IMPLEMENTATION

One way to capitalize on the collinear normal test is as follows. If two patches do *not* contain a collinear normal (and do not turn more than 90°), then there is no closed loop. Denoting the two surfaces by $\mathbf{P}(s, t)$ and $\mathbf{Q}(u, v)$, a collinear normal line satisfies all of the following four equations:

$$\mathbf{P}_s \times \mathbf{P}_t \cdot \mathbf{Q}_u = 0 \tag{1}$$

$$\mathbf{P}_s \times \mathbf{P}_t \cdot \mathbf{Q}_v = 0 \tag{2}$$

$$(\mathbf{P} - \mathbf{Q}) \cdot \mathbf{P}_s = 0 \tag{3}$$

$$(\mathbf{P} - \mathbf{Q}) \cdot \mathbf{P}_t = 0 \tag{4}$$

An efficient way to determine the nonexistence of collinear normals is to use interval arithmetic[10]. The bivariate polynomials $\mathbf{P}$, $\mathbf{Q}$, $\mathbf{P}_s$, $\mathbf{P}_t$, $\mathbf{Q}_u$ and $\mathbf{Q}_v$ are replaced

(From Sederberg, Christiansen, and Katz)

Summary?

Algebraic methods: too expensive, too complicated, too unstable, give too much information!

Homotopy methods: basically the same problems as algebraic methods.

Continuation methods: cheap but unreliable. The problem of making them reliable is arguably as hard as the problem we were solving in the first place.

But wait, we forgot one...

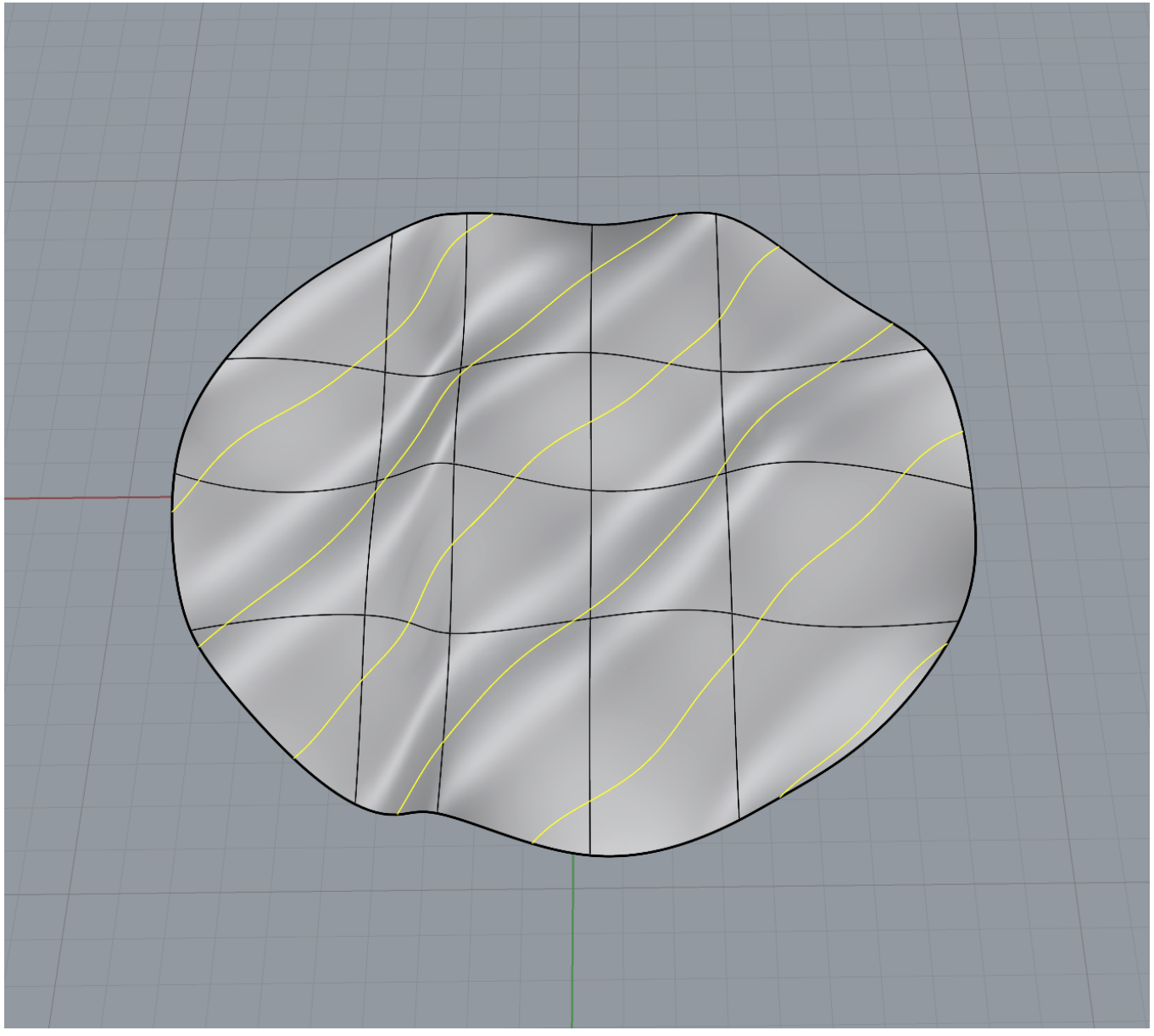
Subdivision methods

“Numerical implicit function theorem”:

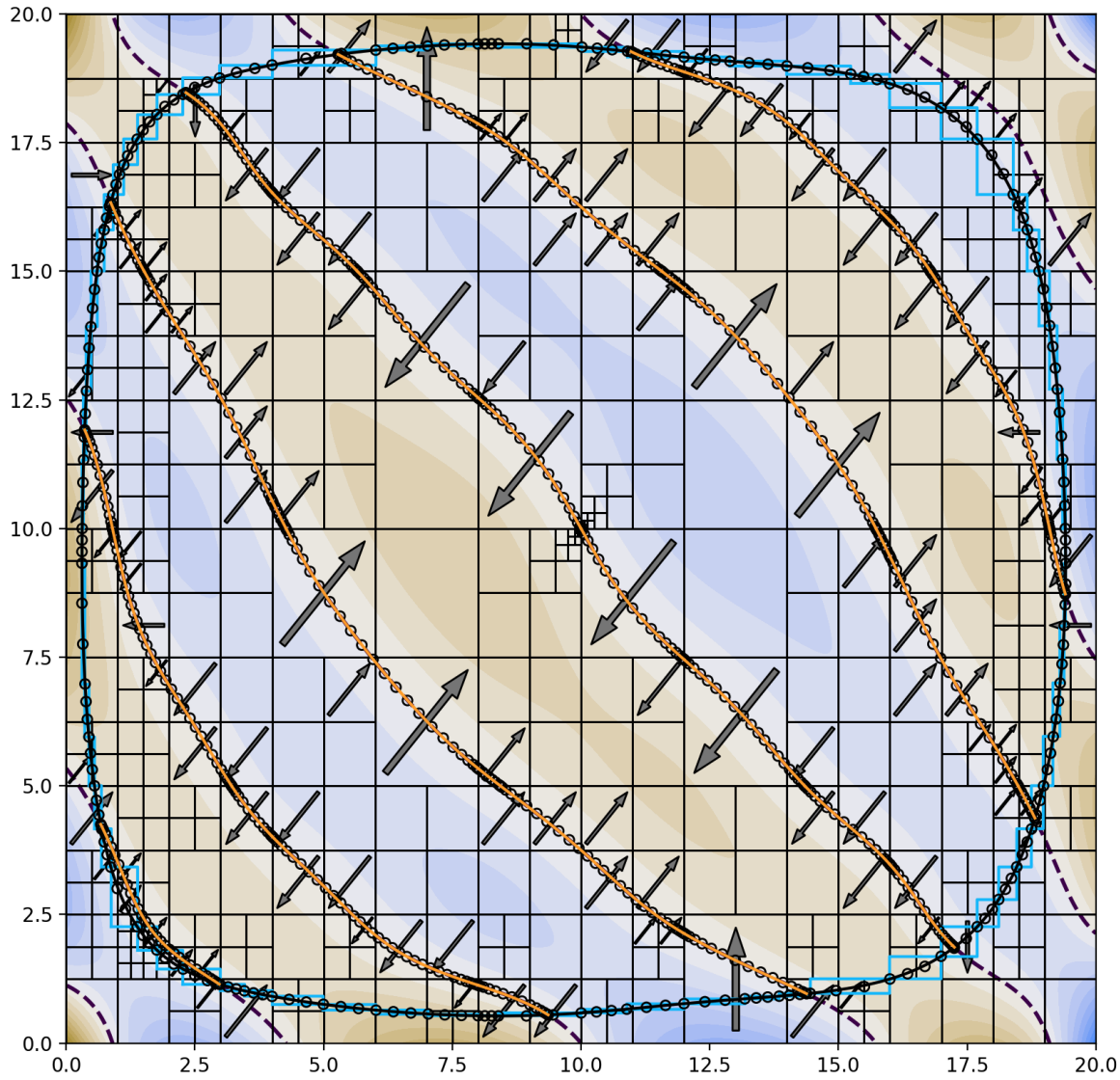
1. Use a quadtree or KD-tree (or both) to recursively subdivide space
2. Over each cell, use interval arithmetic (or similar) to try to prove some analytic property of the level set function, e.g.:
   - Graph of a function in x or y?
   - Node with multiple branches?
   - Range of the level set function small?
   - etc.
3. Do something with “good” boxes; otherwise, terminate when boxes get too small.

Boxes get smaller $\Longrightarrow$ function gets smoother $\Longrightarrow$ interval arithmetic becomes more effective.

Subdivision methods



Subdivision methods



Trimming curves are a pain in the ...!

Subdivision methods

Numerically verified subdivision provides a means of "solving" for global topology **efficiently** and **rigorously**

Once we have computed the global topology, we **approximate** the solution.

Mantra: **first compute the topology**, then **approximate the solution**...

In each of the three other approaches, the two are interleaved, making a systematic approach fraught.

FIRST COMPUTE THE TOPOLOGY...

Subdivision methods

FIRST COMPUTE THE TOPOLOGY...

...THEN APPROXIMATE THE SOLUTION!

Solving $\phi(x, y) = 0$

Compute the topology:

1. Pop rectangle from queue.
2. If rectangle is very tiny, declare it a “small box”.
3. Optional: Krawczyk to check for saddle point.
4. Compute interval inclusion for ϕ, $\nabla\phi$, $\nabla^2\phi$.
5. Use interval inclusions to decide if $\phi(x, y) = 0$ is the graph of a function over rectangle.
6. If not, choose between binary or quad split, insert into queue.

Solving $\phi(x, y) = 0$

Remarks about "computing the topology":

- For general ϕ, impractical to allow subdivision down to floating point resolution. Must have small box handling strategy otherwise too slow.
- Saddles are prevalent, can be easy to find them, powerful topological guarantees if you do.
- Performance ultimately tied to how good interval inclusion is. Need strategy for general smooth ϕ.
- Graph-of-a-function really important for practical implementation. We want to avoid continuation methods!
- We also ensure that each edge is cut in at most one interior point.

Solving $\phi(x, y) = 0$

Approximate the solution:

1. Connect each “rectangle/function graph” into connected chains.
2. Use hybrid secant/bisection rootfinder to evaluate the graph: $x(y)$ or $y(x)$ such that $\phi(x, y) = 0$.
3. Approximate level curve using piecewise Chebyshev “spline” (degree 7 and 8 have been good in practice).
4. Convert this into a G^{d-1} curve, reparametrize by arc length or use connection matrix to fix param and make (approximately) C^{d-1}.
5. Convert to maximally continuous B-spline over same knot span (many options here).

Solving $\phi(x, y) = 0$

Remarks about "approximating the solution":

- The zero level set is analytic almost everywhere in the interior when viewed as the graph of a function.
- Chebyshev approximation of an analytic function ensures exponential order of convergence for function and all of its derivatives (!!!).
- Sufficiently high order Chebyshev panels means we can cheaply get lots of digits in the function and its derivatives just by sampling appropriately.
- Arguably all we're doing is using Chebyshev interpolation for optimal knot insertion.

Handling trim curves

In Rhino, trim curves form a closed, **watertight** (!) loop, that is consistently oriented.

The “main” loop might contain interior loops which have a consistent winding, but no loops inside loops (AFAIU).

ADD PICTURE OF TRIM CURVES FROM RHINO DOCS

Handling trim curves

How we handle them:

- Decompose trimming curves into monotonic segments using interval arithmetic.
- Build BVH on those segments (possibly further subdividing if it makes sense).
- Use for a fast and robust inside-outside test.

Monotonicity + inside-outside $\Longrightarrow$ **additional subdivision rule**: keep going until each function graph crosses trim boundary at most once.

Solving $\phi(x, y) = 0$

Remarks about overall approach

- If we can cover $\phi(x, y) = 0$ with boxes (i.e., small box handling doesn't kick in), if no polynomial panel leaves its box, and if the subsequent B-spline "stays put", we have a high order isotopic parametric approximation of the zero level set.
- So far, "approximate" seems much more expensive than "topology" because I'm computing everything to about 13 digits of accuracy.
- Figuring out how to systematically reduce accuracy comes next.

What’s next?

- This is a pilot project to test feasibility of approach
- Working on replacing backend of Rhino’s `Section`, `Contour`, and `ClippingPlane` commands
- Working last bugs out of implementation, will start optimizing code next
- If successful, will move on to systematically improving other commands: **surface-surface intersection comes next...**
- Silhouette computation probably next after that, Rhino’s `Make2D` command needs love.

Long term?

- **So many** nonlinear contouring problems to solve...
- Use cut cell quadratures for implementation of CutIGA... many useful CAD problems to solve using simulation, CutIGA is an attractive framework. Do it right this time?

Thank you! Questions?